

Modul Pemrograman Berbasis Objek



**Tim Penyusun:
Unit Pengembangan Akademik**

**Universitas Bina Sarana Informatika
Fakultas Teknik dan Informatika
Program Studi Teknologi Informasi (S1)
2026**

BAB I

MENGENAL BAHASA PEMROGRAMAN JAVA

1.1. Kopetensi Dasar

Pada pembahasan Bab 1 ini penulis mengajak mendiskusikan mengenai apa itu bahasa pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa dapat mendeskripsikan penggunaan konsep pemrograman berbasis objek, bisa mendeskripsikan dapat memahami aktifitas dasar pemrograman java.. Penulis berharap, diakhir pembahasan, para pembaca bisa :

- a. Konsep Pemrograman Berbasis Objek
- b. Karakteristik Pemrograman Berbasis Objek.
- c. Mengetahui Program Java dan Membuat Program Java pada Editor Java.
- d. Menerjemahkan dan Menjalankan Program Java
- e. Mengetahui Tipe Data Primitif.
- f. Membuat dan Menggunakan Variabel
- g. Penggunaan Operasi I/O Stream

1.2. Sejarah Singkat Pemrograman Java

Bahasa pemrograman Java yang pada awalnya bernama bahasa pemrograman OAK dipelopori oleh James Gosling, Patrick Naughton, Chris Warth, Ed Frank dan Mike Sheridan dari Sun Microsystem, Inc pada tahun 1991, ketika perusahaan tersebut memulai Green Project (proyek penelitian membuat bahasa pemrograman yang dapat berjalan diatas berbagai *platform*).

Hal ini merupakan sebuah terobosan yang cukup besar sebab sebelumnya aplikasi-aplikasi yang dikembangkan untuk system operasi (dan perangkat keras) tertentu hanya akan bias berjalan dengan baik diatas system operasi tertentu pula. Dengan kata lain, dimasa lalu ,aplikasi yang ditulis diatas sebuah system operasi (misalnya Windows)hampir mustahil bias dijalankan di atas system operasi lain(contoh Unix/Linux).

Upaya yang dilakukan James Gosling dari Sun Microsystem ini cukup berhasil. Bahasa pemrograman Java saat ini bias digunakan untuk mengembangkan aplikasi-aplikasi desktop (menggunakan JSE[Java Standard Edition]), aplikasi-aplikasi yang berjalan dilingkup enterprise dan internet(JEE[Java Enterprise Edition]), serta aplikasi-aplikasi yang berjalan disarana-sarana komputasi yang terbatas sumberdaya pemrosesan dan terbatas sumber daya penggunaan memorinya(JME[Java Micro Edition]), contohnya telepon seluler,PDA, dan sebagainya.

Versi awal Java ditahun 1996 sudah merupakan versi release sehingga dinamakan Java Versi 1.0. Java versi ini menyertakan banyak paket standar awal yang terus dikembangkan pada versi selanjutnya:

- java.lang: Peruntukan kelas elemen-elemen dasar.
- java.io: Peruntukan kelas *input* dan *output*, termasuk penggunaan berkas.
- java.util: Peruntukan kelas pelengkap seperti kelas struktur data dan kelas kelas penanggalan.
- java.net: Peruntukan kelas TCP/IP, yang memungkinkan berkomunikasi dengan komputer lain menggunakan jaringan TCP/IP.
- java.awt: Kelas dasar untuk aplikasi antarmuka dengan pengguna (GUI)
- java.applet: Kelas dasar aplikasi antar muka untuk diterapkan pada penjelajah web.

1.3. Karakteristik Pemrograman Java

Sifat-sifat pemrograman java antara lain : berorientasi objek, terdistribusi, multiplatform, dan multithread.

A. Java berorientasi Objek

Penulisan program harus dibuat dalam bentuk objek-objek,kemudian memodelkan sifat masing-masing objek dalam program. Java menentukan dan mengatur interaksi antar objek.

B. Java bersifat terdistribusi

Java dapat digunakan untuk menyebarkan /mendistribusikan data atau informasi dalam workstation client,email server,database server, dan webserver(dalam jaringan komputer lokal dan internet).

C. Java bersifat multiplatform

Program java yang sudah diterjemahkan /dikompilasi dengan benar dan berhasil dapat dijalankan pada platform yang lain atau pada sistem operasi yang lain.

Java menggunakan prinsip WORA(Write Once and Run Anywhere). Sekali menuliskan program maka hasil programnya dapat dijalankan pada sistem operasi mana saja.Artinya sekali menulis program dalam bahasa Java dan mengkompilasinya maka program tersebut dapat dijalankan pada berbagai system operasi, seperti DOS, Windows, Linux, MacOS, Solaris

D. Java Bersifat Multithread

Adalah banyaknya pekerjaan yang dikerjakan dalam satuan waktu yang hampir bersamaan. Sebagai contoh, suatu web server mempunyai ratusan client server—beberapa client yang mengakses web server secara bersamaan. Jika web server tersebut hanya mempunyai satu thread, berarti client yang lain harus menunggu. Oleh karena itu di butuhkan multithread untuk dapat melayani client server.

1.4. Konsep Pemrograman Berbasis Objek

Pada konsep pemrograman berbasis objek ini, kita akan membahas mengenai objek, Class dan Method. Berikut penjelasan singkat dan padat mengenai ketiga konsep diatas.

1.4.1 Object dan Class

Dalam Pemrograman Berorientasi Objek melihat atau memandang sesuatu berdasarkan objek. Objek sebenarnya mencerminkan pola kerja manusia dalam kehidupan sehari-hari.

Pada suatu objek dapat dilihat menjadi 2 (dua) hal, yaitu :

a. Atributte

Atribut merupakan segala sesuatu yang melekat pada Object. Didalam penerapan didalam program, atribut adalah Variabel atau Member.

Misalkan pada Object Burung. Atribut-atribut yang melekat pada burung, misalnya paruh, ekor, sayap, kaki, mata, dan lain-lain.

b. Behaviour

Behaviour merupakan pola tingkah laku atau perilaku yang dimiliki oleh objek. Misalnya pada objek Burung memiliki perilaku diantaranya terbang, mengepakkan sayap, berjalan dan lain-lain.

Didalam penerapan didalam program, Behaviour adalah Method atau Fungsi.

Bentuk penulisan Class, seperti dibawah ini :

```
[public | private] [abstract] Class Nama_Class
{
... daftar property...
... daftar Method ...
}
```

Bentuk penulisan mendeklarasikan Object, dengan menggunakan new, seperti dibawah ini :

```
nama_Class nama_objek = new nama_Class();
nama_Class, merupakan nama Class yang akan dijadikan objek.
```

nama_objek, merupakan nama objek baru.

Contoh pembuatan Class sederhana :

```
Class burung
{
String jenis, warna;
int usia;
}
Class burung_terbang
{
public static void main(String[] args)
{
//membuat objek
burung burung_elang = new burung();
.....
.....
}
```

1.4.2 Method

Method adalah implementasi operasi yang bisa dilakukan oleh Class dan Object. Operasi-operasi yang dilakukan oleh Methode, diantaranya, yaitu :

1. Suatu Method bisa menerima dan memanipulasi data atau field didalam diri Method tersebut.
2. Suatu Method bisa mempengaruhi nilai suatu Object lain.

Berikut bentuk penulisan deklarasi Method:

```
Tipe_Akses Tipe_Return NamaMethod(Argumen1, Argumen2,...,Argumen-N)
{
... Badan / Tubuh Method ..
}
```

Berikut penjelasan deklarasi Methode diatas :

1. Tipe Akses, menyatakan tingkatan akses untuk memproteksi akses terhadap data-data didalam Method, tipe akses ini bersifat opsional.
2. Tipe Return, menyatakan nilai hasil yang diolah oleh Method akan dikembalikan atau akan mengirimkan kepada objek yang memanggil Method. Bentuk Tipe Return, bisa berupa tipe data primitive yaitu integer, float, double dan lain-lain.

Apabila Method tidak akan mengembalikan nilai kepada objek yang memanggilnya, maka bisa dituliskan didepan nama Method dengan perintah void.

1.5. Karakteristik Pemrograman Berbasis Objek

Sekarang ini dalam tahap mempelajari pemrograman berbasis objek, anda harus mengenal karakteristik yang dimiliki pemrograman berbasis objek. Adapun ketiga karakteristik tersebut, yaitu :

a. Enkapsulasi (Encapsulation)

Karakteristik ini merupakan suatu cara bagaimana menyembunyikan sedemikian rupa suatu proses kedalam sistem, hal ini berguna untuk menghindari interferensi dari luar sistem dan juga lebih untuk menyederhakan sistem itu sendiri.

Kita ambil contoh, pada saat anda mengganti chanel TV menggunakan remote TV, apakah anda mengetahui proses yang terjadi didalam TV tersebut ?, maka jawabannya tidak tau, dan anda pun sebagai pembeli TV tidak mau dipusingkan dengan proses yang terjadi. Maka hal tersebut menyederhakan sistem.

b. Pewarisan (Inheritance)

Pewarisan, bahasa kerennya Inheritance. Dalam pemrograman berbasis objek, dimungkinkan suatu Class bisa mewariskan atribut dan Method kepada Class yang lainnya atau subClass, sehingga membentuk Class hirarki.

Sebagai contoh, pada saat kita bicara mengenai bus, maka bus tersebut bisa mewariskan kepada bus yang lain berupa, nomor trayek, body besar, jumlah penumpang banyak dan lain sebagainya.

c. Polymorphism

Karakteristik dari polymorphism yaitu memungkinkan suatu objek dapat memiliki berbagai bentuk atau banyak bentuk. Bentuk dari objek ini bisa sebagai Object dari Classnya sendiri atau Object dari superClassnya.

Pada polymorphism kita akan sering menjumpai 2 (dua) istilah yang sering digunakan dalam pemrograman berbasis objek, istilah tersebut yaitu :

a). Overloading.

Overloading yaitu menggunakan 1 (satu) nama objek untuk beberapa Method yang berbeda ataupun bisa juga beda parameternya.

b). Overriding

Overriding akan terjadi apabila ketika pendeklarasian suatu Method subClass dengan nama objek dan parameter yang sama dengan Method dari superClassnya.

d. Abstrak

Abstrak didalam pemrograman berbasis objek, yaitu dimaksudkan untuk melihat suatu sistem, menjadi lebih sederhana atau simple. Apabila kita melihat suatu sistem, misalnya motor, maka bisa kita lihat ada apa saja disistem motor ?, yang pasti ada sistem pengapian, sistem rem, sistem oper gigi dan lain sebagainya. Maka kesemua sistem-sistem tersebut kalau kita lihat menjadi satu sistem yang lebih sederhana yaitu sistem motor.

e. Modularity

Setiap objek didalam pemrograman berbasis objek, memungkinkan bisa dituliskan atau dibuat secara terpisah-pisah dari objek lainnya. Sehingga program bisa lebih mudah dikembangkan dan dimodifikasi.

Kita ambil contoh pada sistem motor, bisa anda bayangkan seandainya sistem rem tersebut langsung menyatu pada objek utama pada motor, apabila seandainya ada perbaikan atau mengubah, maka akan membongkar objek utamanya, baru keobjek tujuan, maka hal ini akan makan waktu yang lama. Maka dengan adanya modularity, apabila ada objek yang akan diperbaiki atau dimodifikasi, langsung keobjek tujuannya saja.

1.6. Jenis Program Java

Program Java dapat dibedakan menjadi dua jenis, yaitu applet dan aplikasi :

a.Applet

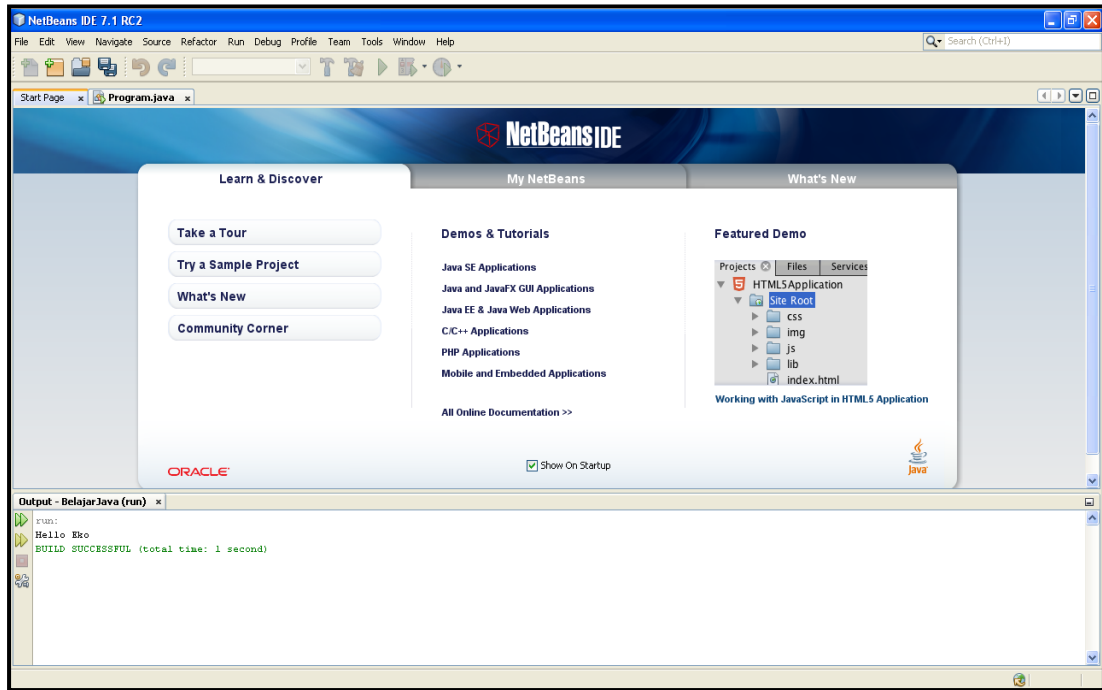
Applet adalah program yang dibuat dengan Java, dapat diletakan di web server dan di akses melalui web browser. Dalam hal ini browser yang digunakan adalah yang memiliki kemampuan Java (misalnya Netscape Navigator, Internet Explorer, Mozilla Firefox).

b.Aplikasi

Aplikasi adalah program yang dibuat dengan Java yang bersifat umum. Aplikasi dapat dijalankan secara langsung,tidak perlu perangkat lunak browser untuk menjalankannya. Aplikasi dapat anda bayangkan seperti program yang anda tulis dengan bahasa c atau pascal. Setelah di kompilasi, anda dapat mengeksekusinya secara langsung.

1.7. Mengenal Program Java

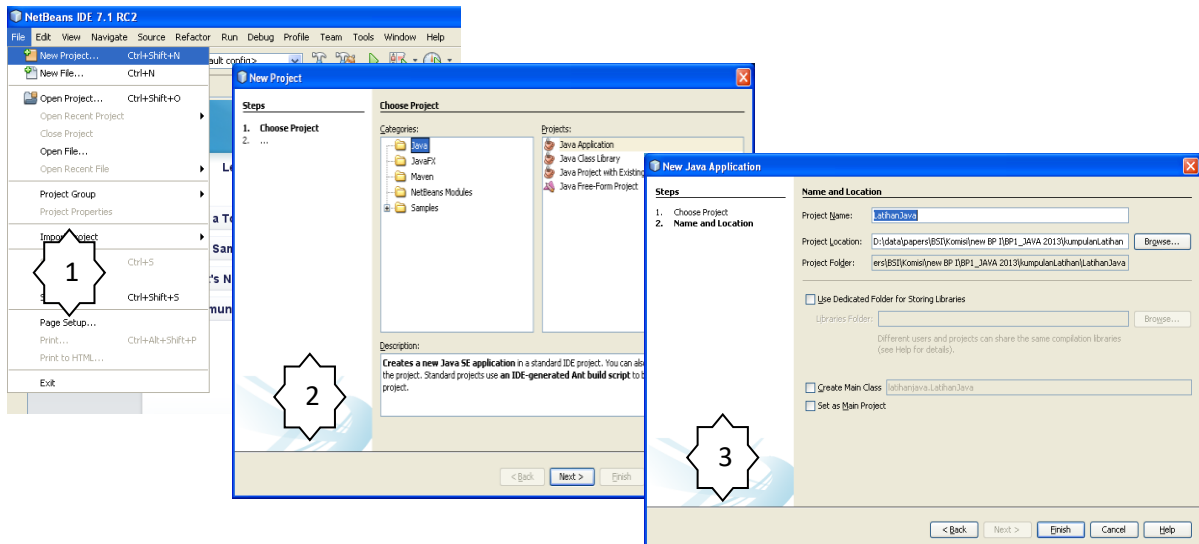
Dalam subbab ini, kita akan membuat dan mengenal program java yang ada sekarang ini. Seperti yang sudah di jelaskan diatas cara umum, program java dibagi menjadi 2(dua) macam, yaitu Java Application dan Java Applet. Lebih lanjut akan dibahas pada penjelasan dibawah ini dan sekaligus diterapkan pada editor Java. Untuk saat ini penulis menggunakan editor Netbeans versi 7.0 yang bisa didownload secara gratis. File Program Java merupakan File Program yang dapat dikompilasi, dan dijalankan untuk menampilkan hasilnya serta mempunyai ekstensi .java.



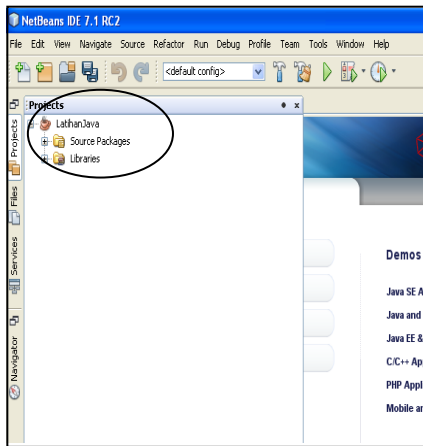
Gambar 1. Tampilan awal NetBeans

1.7.1. Langkah pembuatan Java Application

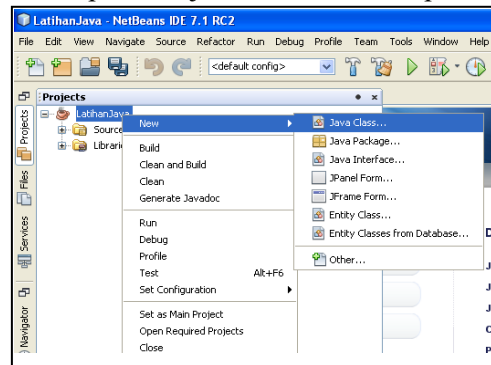
- 1).Klik File -> New Project
- 2).Kemudian akan Tampil Jendela New Project, pilihJava Pada Categories dan Java Application Pada Project, kemudian klik next.
- 3).Setelah itu akan tampil Jendela New Java Application, tentukan nama Project dan juga Folder penyimpanan, Kemudian Klik Finish.



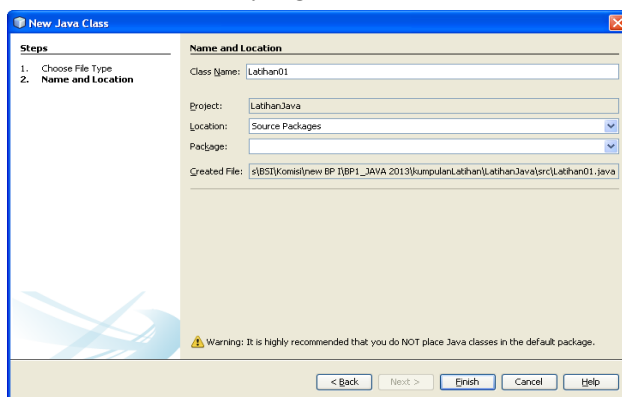
4).Selanjutnya akan tampil Project yang dibuat pada layar sebelah kanan di jendela Project.



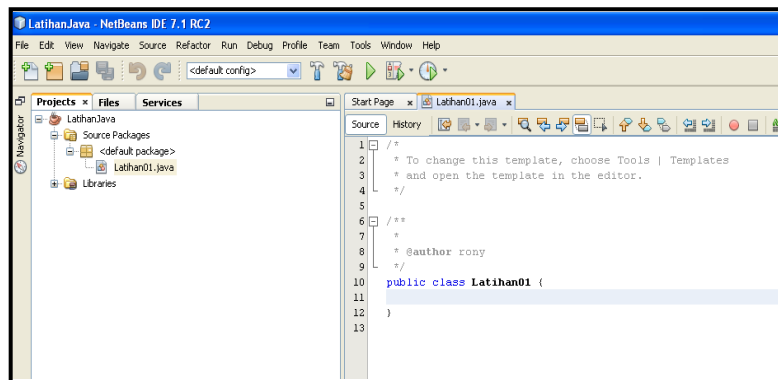
5).Selanjutnya untuk membuat sebuah File baru, Klik kanan pada Project LathianJava -> pilih new -> Java Class



6).Ketikan nama Class yang akan di buat, kemudian finish.



7).Setelah berhasil membuat sebuah file/Class maka akan tampil jendela pengetikan program yang akan dibuat.



Selanjutnya anda dapat mengetikan kode-kode Program seperti dibawah ini:



Setelah kode program di ketikan maka langkah selanjutnya adalah meng-compile atau mengeksekusi kode program dengan memilih run pada menu bar kemudian pilih run file(shift+f6).

Berikut penjelasan mengenai contoh program diatas.

1. Class Latihan01, merupakan mengawali pendefenisian Class, dan diikuti dengan nama Class yaitu : Latihan01.
2. main adalah metode yang digunakan untuk mengawali segala bentuk eksekusi pada program java. Metode main ini, didefinisikan sebagai public static void, yang memiliki arti ;
 - a. public, yang berarti metode ini bisa dipanggil dan digunakan didalam Class atau diluar Class.
 - b. static, yang berarti memiliki sifat yang sama disemua instant Class.
 - c. void, yang berarti bahwa metode ini tidak mengirimkan nilai balik
3. Didalam metode main, terdapat String[] args, yang memiliki pengertian ;
 - a. String[], adalah tipe data objek yang menangani serangkaian karakter-karakter yang berjenis array.
 - b. args, adalah variabel objek.
4. System.out.println("Selamat Belajar Java Programming"); , yang memiliki pengertian;
 - a. System.out, adalah stream yang digunakan untuk menangani keluaran standar java.
 - b. println(); , merupakan metode yang digunakan untuk menampilkan teks dilayar.
 - c. Tanda titik koma (;), digunakan untuk mengakhiri pernyataan.

1.7.2. Langkah pembuatan Java Applet

Pada langkah aawal pembuatan Applet hampir sama dengan pembuatan class/file baru

1. Buatlah sebuah class dengan nama JavaApplet.java dengan kode program seperti di bawah.

```
1  import java.applet.*;
2  import java.awt.*;
3
4
5  /**
6   *
7   * @author rony
8   */
9  public class JavaApplet extends Applet {
10     public void gambar(Graphics g)
11     {
12         g.drawString("Hallo World", 20, 20);
13     }
14 }
15
```

Berikut penjelasan mengenai contoh program diatas.

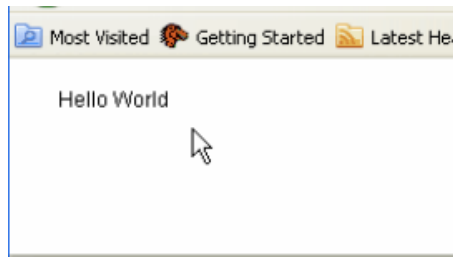
1. public Class JavaApplet extends Applet, merupakan mengawali pendefenisian Class, dan diikuti dengan nama Class yaitu : JavaApplet. Bersifat public yang merupakan turunan dari Applet.
2. public void paint(Graphics g) adalah menggunakan metode paint untuk menggambar semua graphic applet didrawing area, dengan parameter Class abstrak untuk merepresentasikan area applet.
3. g.drawString("Hello World", 20, 20) adalah untuk mencetak text Hello Word, pada posisi baris 20 dan posisi kolom 20.

2. Kemudian buatlah sebuah file html dengan menggunakan notepad simpan dengan nama BrowseApplet.html dengan kode program seperti di bawah ini.

```
BrowseApplet - Notepad
File Edit Format View Help
<html>
<head>
<applet
  code="JavaApplet.class"
  height=300 width=300>
</applet>
</head>
</html>
```


Jalankan file BrowseApplet.html Menggunakan Browser
 Klik menu File pada web browser | klik dan pilih OpenFile
 Arahkan ke nama file yang terdapat file yang telah anda buat diatas. Sebagai contoh

file:///c:/LATIHANJAVA/BrowseApplet.html
 dan hasilnya akan tampak seperti di bawah ini



1.8. Aktifitas Dasar Pemrograman Java

A. Mengetahui Tipe Data

Java memiliki delapan tipe data primitif, meliputi 4 tipe untuk bilangan bulat, 2 tipe untuk bilangan pecahan dan sisanya untuk karakter dan Boolean.

Tipe	Panjang (bit)	Kategori
boolean	8	=
char	16	=
byte	8	Integer
short	16	
int	32	
long	64	
float	32	Floating-point
double	64	

Tabel 1. Tipe Data Primitif pada Java

Berikut kita bahas secara singkat dan padat mengenai keempat kategori tipe data diatas.

1. Tipe Bilangan Bulat (*Integer*)

Ada empat macam tipe bilangan bulat. Perbedaan masing-masing tipe data terletak pada jangkauan nilai yang dicakup. Tipe data integer digunakan untuk operasi data bilangan bulat dan perhitungan aritmatika. Berikut keempat tipe data yang tercakup kedalam kategori integer.

Tipe Data	Ukuran	Jangkauan
byte	8 bit	-128 sampai 127
short	16 bit	-32.768 sampai 32.767
int	32 bit	-2.147.483.648 sampai 2.147.483.647
long	64 bit	-9.223.372.036.854.775.808 sampai 9.223.372.036.854.775.807

2. Tipe Bilangan Pecahan (*Floating Point*)

Floating-point dasarnya digunakan ketika kita mempunyai situasi dimana mendapatkan hasil atau output dalam bentuk desimal dan seluruh angka yang tidak disebutkan dalam tipe data integers. Dua tipe data yang berkaitan dengan bilangan pecahan adalah float dan double.

Tipe Data	Ukuran	Jangkauan
float	32 bit, presisi 6-7 digit	-3.4E38 s.d +3.4E38
double	64 bit, presisi 14-15 digit	-1.7E308 s.d 1.7E308

3. Tipe Data karakter (*Char*)

Tipe data Character digunakan untuk mendefinisikan sebuah karakter yang merupakan simbol dalam karakter Set, seperti huruf dan angka. Keyword tipe data Character ini yaitu char, dengan ukuran 16 bit.

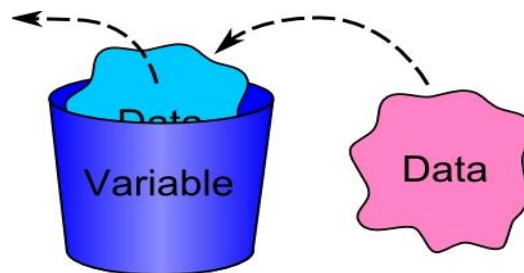
4. Tipe Data Boolean

Tipe data Boolean digunakan untuk menangani keadaan logika atau keadaan dengan dua kemungkinan nilai. Nilai yang dapat digunakan adalah true(benar) dan False(salah).

Selain tipe data – tipe data Primitive yang dimiliki oleh Java. Java memiliki tipe data class Object. Tipe data class Object yang sering digunakan yaitu String. String disediakan untuk menampung sejumlah character.

B. Variabel

Variabel adalah suatu tempat menampung data atau konstanta dimemori yang mempunyai nilai atau data yang dapat berubah-ubah selama proses program. Atau bisa juga di jelaskan Variabel menyatakan suatu lokasi di dalam memori computer yang digunakan untuk menyimpan suatu nilai dan nilai yang ada di dalamnya bias diubah. Variabel dapat di bayangkan suatu tempat, tempat dapat diisi pada suatu saat dan pada saat lain dapat di ubah.



Gambaran Variabel.

Dalam pemberian nama variabel, mempunyai ketentuan-ketentuan antara lain Tidak boleh ada spasi (cth : gaji bersih) dan dapat menggunakan tanda garis bawah (_) sebagai penghubung (cth : gaji_bersih). Tidak boleh diawali oleh angka dan menggunakan operator aritmatika. Pembacaan variable dalam pemrograman java bersifat case sensitive.

1. Mendeklarasikan Variabel

Deklarasi Variabel

Deklarasi Variabel adalah proses memperkenalkan variabel kepada java dan pendeklarasian tersebut bersifat mutlak karena jika tidak diperkenalkan terlebih dulu maka java tidak menerima variabel tersebut. Deklarasi Variabel ini meliputi tipe variabel, seperti : integer atau character dan nama variabel itu sendiri. Setiap kali pendeklarasian variabel harus diakhiri oleh tanda titik koma (;).

Bentuk penulisannya :

Tipe data [nama_variabel];

Contoh Deklarasi :

```
String nama_mahasiswa;
char grade;
```

```
float rata_rata ;  
int nilai1, nilai2;
```

2. Menempatkan Nilai kedalam Variabel

Setelah pendeklarasian Variabel dilaksanakan, selanjutnya variabel tadi bisa anda masukan nilai kedalam variabel. Berikut cara yang mudah untuk menempatkan nilai kedalam variabel.

Berikut Bentuk penulisannya :

```
nama variabel = nilai;
```

Contoh Penempatan Nilai kedalam Variabel :

```
nama_mahasiswa = "Irvan Y. Ardiansyah";  
grade = 'A';  
rata_rata = 95.75;  
nilai1 = 90; nilai2 = 95;
```

Java bisa juga memperbolehkan memberikan nilai yang sama kebeberapa nama variabel yang berbeda. Seperti contoh dibawah ini:

```
a = c = d = 7;
```

Pada contoh diatas variabel a, c, dan d masing-masing berisi nilai 7.

C. Konstanta

Konstanta merupakan variabel yang tidak dapat diubah lagi nilainya. Di Java pembuatan konstanta dapat menggunakan kata kunci final pada awal deklarasi variabel.

Contoh :

```
final int nilaiAkhir = 100;
```

D. Membuat Komentar

Pada bahasa pemrograman manapun komentar program biasa digunakan untuk memberikan penjelasan baris atau blok program supaya pembaca program atau programmer lainnya supaya bisa mengerti bagian-bagian program tersebut. Ada 3 (tiga) cara memberikan komentar program pada Java, yaitu :

a. End Of Line Comment (//)

Komentar dengan tanda slash ganda (//) disebut dengan end-of-line comment, karena semua perintah program, komentar-komentar atau penjelasan program berada setelah tanda slash ganda, semua dianggap sebagai komentar dan komentar hanya satu baris saja. Sebagai contoh:

```
// isi komentar program  
// a = b + c;
```

b. Multiple Line Comment (/* */)

Komentar dengan tanda slash dan asterik (/ * */) disebut dengan Multiple-Line Comment, karena ini, perintah program, komentar-komentar atau penjelasan program berada dalam apitan tanda slash dan asterik, semua dianggap sebagai komentar, dan komentar bisa lebih dari satu baris. Sebagai contoh :

```
/* isi komentar program bisa  
terdiri dari beberapa baris komentar  
atau informasi */
```

c. Javadoc Comment (/** */)

Komentar dengan tanda slash ganda didepan dan asterik (/** */) disebut dengan Javadoc Comment. Penggunaannya sama seperti Multiple Line Comment, akan tetapi penggunaannya untuk dokumentasi-dokumentasi didalam program. Sebagai contoh : Semua komentar program atau penjelasan program pada saat program java dicompile tidak ikut serta dicompile, karena tidak dianggap sebagai suatu baris program. Kode program contoh penggunaan Variabel, Konstanta dan Komentar

```

1  /*-----
2      Nama Berkas   :LatihanVar.Java
3      Dibuat Oleh   :Rony, 2013
4      Contoh Penggunaan Variabel,Konstanta dan Komentar
5  -----*/
6  public class LatihanVar {
7
8      public static void main(String[] args) {
9          // TODO code application logic here
10
11          int JariJari;
12          double keliling;
13          final double PI=3.14;
14
15          JariJari = 30;
16          |
17          keliling= 2*PI*JariJari;
18          System.out.println("Keliling Lingkaran = "+keliling);
19          System.out.println("=====");
20      }
21
22  }

```

Setelah diJalankan berikut tampilan Output

```

Output - LatihanJava (run)
run:
Keliling Lingkaran = 188.4
=====
BUILD SUCCESSFUL (total time: 1 second)

```

BAB II

TEORI DAN KONSEP PEMROGRAMAN BERBASIS OBJECT

2.1. Kopetensi Dasar

Pada pembahasan Bab 2 ini penulis mengajak mendiskusikan mengenai penggunaan operator-operator yang disediakan oleh Bahasa Pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa/i atau pembaca bisa mendeskripsikan dapat memahami penggunaan operator-operator pada bahasa pemrograman Java. Penulis berharap, diakhir pembahasan, para pembaca bisa :

- Penggunaan Operator Aritmatika.
- Penggunaan Operasi Pemberi Nilai
- Penggunaan Operator Penambah dan Pengurang
- Penggunaan Operator Logika

2.2. Konsep Pemrograman Berorientasi Objek

Pemrograman berorientasi Objek yang populer pada masa kin menjadikan objek sebagai komponen utama dalam program. Objek menggabungkan data dan fungsi sebagai suatu kesatuan. Sedangkan pada pemrograman yang tidak berorientasi objek, fungsilah yang menjadi perhatian utama. Pendekatan pemrograman berorientasi objek membuat pengembangan program dapat dilakukan dengan lebih mudah, mengurangi duplikasi kode dan mengurangi kesalahan.

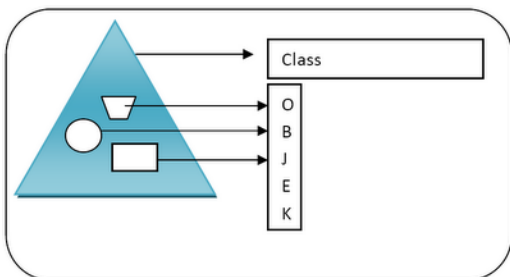
Pada pemrograman berorientasi objek terdapat dua istilah yang sangat terkenal yaitu kelas dan objek. Kedua istilah ini sering di pertukarkan. Untuk lebih memudahkan pemahaman tentang objek, saya beri sebuah contoh : Meja, Kursi, Jam Dinding dan yang lainnya merupakan contoh objek. Objek yang disebutkan tadi umumnya berada didalam sebuah ruangan, ruangan apabila kita melihatnya dari sebuah rumah bisa juga dikategorikan sebuah objek dan rumah jika dilihat dari cara pandang yang lebih luas lagi juga merupakan objek. Jadi sebuah Objek merupakan suatu bentuk atau model yang tergantung dari cara pandang objek dimaksud.

Meja, Kursi, Jam Dinding ada dalam sebuah Ruangan.

Ruang A, Ruang B, dan Ruang yg lain berada dalam sebuah Rumah.

Rumah 1, Rumah 2 dan Rumah yang lain juga berada didalam suatu tempat yang lebih luas yaitu Bumi, Dst.

Dan bukan tidak mungkin didalam Objek Jam Dinding terdapat objek-objek yang lain yang lebih kecil misalnya Baterai, Gear, dll. Cara pandang kita dari sebuah Ruangan, Rumah atau Bumi merupakan Class apabila kita melihatnya dalam Pemrograman.



Setiap objek memiliki Atribut dan Method. Atribut adalah segala sesuatu yang berhubungan dengan karakteristik objek. Seperti Tipe, Model, Pintunya, Kusennya menggunakan kayu apa?, Warna Catnya, merupakan salah satu atribut dari Rumah. Sedangkan Method dari Rumah adalah dapat melindungi dari panas, hujan, angin. Jadi Method merupakan Fungsi atau segala sesuatu yang dapat dilakukan Objek.

Setiap objek memiliki Atribut dan Method. Atribut adalah segala sesuatu yang berhubungan dengan karakteristik objek. Seperti Tipe, Model, Pintunya, Kusennya menggunakan kayu apa?, Warna Catnya, merupakan salah satu atribut dari Rumah. Sedangkan Method dari Rumah adalah dapat melindungi dari panas, hujan, angin. Jadi Method merupakan Fungsi atau segala sesuatu yang dapat dilakukan Objek.

Untuk lebih memahami konsep class dan object mari kita coba latihan

Buatlah sebuah Class dengan nama RumahKu.java

```
1
2 public class RumahKu { //Buat class Rumah
3
4 //buat method static Meja
5 public static void Meja()
6 {
7     System.out.println("Ada Meja");
8 }
9 //method jam dinding
10 void JamDinding()
11 {
12     System.out.println("Ada Jam Dinding");
13 }
14
15 }
16
```

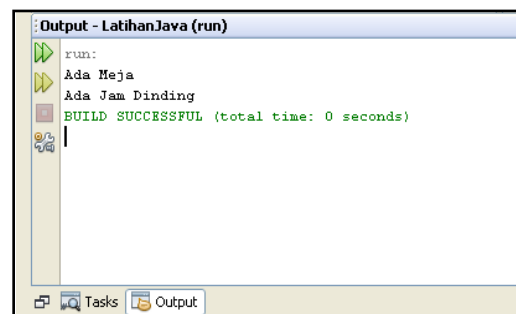
Compile kode program diatas, dan tidak akan ada output dari kode program tersebut

Kemudian buatlah sebuah Class dengan Nama isiRumahKu

```
1
2 public class isiRumahKu {
3     public static void main(String[] args)
4
5 {
6     Rumahku.Meja();
7     Rumahku Panggil = new Rumahku();
8     Panggil.JamDinding();
9 }
10 }
```

Setelah kode program diatas selesai di ketik kemudian jalankan kode program dengan memilih **run** pada menu bar setelah itu pilih **run file**.

Tampilan Output kode Program isiRumahKu.java



```
Output - LatihanJava (run)
run:
Ada Meja
Ada Jam Dinding
BUILD SUCCESSFUL (total time: 0 seconds)
```

Penjelasan Program

Pada latihan kedua ini kita telah membuat 2 (dua) buah file, yaitu :

1. RumahKu.java _ membuat sebuah class dengan nama “RumahKu.class”
2. isiRumahKu.java _ class yang lain “isiRumahKu.class” yang menggunakan class RumahKu

Pada file pertama “RumahKu.java” diawali dengan membuat sebuah class public “Rumah” agar dapat digunakan pada class yang lain, didalam class “Rumah” terdapat 2 (dua) buah method yaitu “Meja” dan “JamDinding”, perhatikan dari cara penulisannya!
Method “Meja”

```
public static void Meja()  
{ System.out.println("Ada Meja"); }
```

Kata kunci “public static” digunakan agar method ini mempunyai jangkauan global dan isinya tetap, sedangkan Method “JamDinding” hanya digunakan untuk jangkauan lokal saja

```
void JamDinding()  
{  
System.out.println("Ada Jam Dinding");  
}
```

Pada file yang kedua “isiRumahKu.java” diawali juga dengan membuat sebuah class public “isiRumahKu” didalamnya terdapat method utama “main” dengan Atribut yang digunakan “String[]args” fungsi kode program ini adalah memanggil class “RumahKu” untuk digunakan pada class “isiRumahKu”, perhatikan cara pemanggilan yang berbeda dari Method yang ada pada class “Rumah”!

```
Rumah.Meja();
```

Pada baris ini, karena Method “Meja” menggunakan kata kunci “public static” maka pemanggilannya dapat secara langsung tanpa harus membuat satu objek baru untuk dapat menggunakan/memanggil method pada class lain, seperti method “JamDinding” pada class “RumahKu”

```
RumahKu Panggil = new RumahKu();
```

Pada baris ini dibuat satu objek baru dengan nama “Panggil” yang merupakan nama lain dari class “RumahKu”, cara memanggilnya adalah seperti pada baris berikut
Panggil.JamDinding();

MUDAH KAN.....!!!!!!

>>> LATIHAN BUATLAH CONTOH KASUS YANG LAIN DENGAN MENERAPKAN KONSEP CLASS DAN OBJECT SEPERTI CONTOH DI ATAS

2.3. Penerapan Karakteristik Pemrograman Berorientasi Objek

a. Enkapsulasi (Encapsulation)

Karakteristik ini merupakan suatu cara bagaimana menyembunyikan sedemikian rupa suatu proses kedalam sistem, hal ini berguna untuk menghindari interferensi dari luar sistem dan juga lebih untuk menyederhakan sistem itu sendiri.

Kita ambil contoh, pada saat anda mengganti chanel TV menggunakan remote TV, apakah anda mengetahui proses yang terjadi didalam TV tersebut ?, maka jawabannya tidak tau, dan anda pun sebagai pembeli TV tidak mau dipusingkan dengan proses yang terjadi. Maka hal tersebut menyederhakan sistem. Di dalam OOP, encapsulation dapat dilakukan dengan terlebih dahulu memahami **access modifier** yang mendefinisikan bagaimana suatu data atau method dapat diakses. Ada empat macam access modifier pada OOP, yaitu:

- Private : hanya diakses class itu sendiri
- Public : dapat diakses dari manapun
- Protected : hanya dapat diakses dari package (satu folder) dan subclass

- Default : tanpa modifier, hanya bisa diakses dari package dan class itu sendiri.

Tabel Wilayah Modifier Akses				
Wilayah Akses	<i>public</i>	<i>protected</i>	<i>default</i>	<i>private</i>
Di kelas yg sama	√	√	√	√
Beda kelas, di package yg sama	√	√	√	x
Beda kelas, beda package, di kelas turunan	√	√	x	x
Beda kelas, beda package, tidak di kelas turunan	√	x	x	x

Contoh Sederhana

1. Buatlah sebuah Class dengan nama **Belajar.Java**

```
class Belajar
{
public String x ="Pintar";
private String y = "Java";
}
```

2. Buatlah sebuah Class dengan nama Pintar.java

```
public class Pintar{
public static void main(String[] args){
Belajar panggil = new Belajar();
System.out.println("Panggil X : "+panggil.x);
System.out.println("Panggil Y : "+panggil.y);
}
}
```

Penjelasan :

Fungsi **public** yang terdapat dalam *class* Belajar pada variable x, memungkinkan nilai dari variable x dapat diakses oleh *class* Pintar. Sedangkan variable y yang menggunakan fungsi **private** tidak dapat dipanggil didalam *class* Pintar.

b. Pewarisan (Inheritance)

Inheritance (pewarisan/penurunan) merupakan penurunan sifat (atribut dan metod) dari induk class kepada subclass atau anak class.

Untuk mendeklarasikan suatu class sebagai subclass atau class turunan dari suatu class dilakukan dengan menambah kata kunci “**extends**”

Contoh :

```
public class B extends A {
.....
}
```

Artinya : class B merupakan subclass atau turunan dari A (class B mewarisi atribut dari class A)

Contoh Penerapan Pewarisan pada program

1. buatlah Class dengan nama sepeda.java

```
5 package pewarisan;
6
7 /**
8  *
9  * @author rony
10 */
11
12 @
13 class Sepeda{
14     int kecepatan = 0;
15     int gir = 0;
16     // method
17     void ubahGir(int pertambahanGir) {
18         gir= gir+ pertambahanGir;
19         System.out.println("Gir:" + gir);
20     }
21     void tambahKecepatan(int pertambahanKecepatan) {
22         kecepatan = kecepatan+ pertambahanKecepatan;
23         System.out.println("Kecepatan:" + kecepatan);
24     }
25 }
```

2. buatlah Class dengan nama Sepeda Gunung.java

```
5 package pewarisan;
6
7 /**
8  *
9  * @author rony
10 */
11
12 class SepedaGunung extends Sepeda {
13     int setSadel;
14     public void setSadel(int nilaiSadel) {
15         setSadel = nilaiSadel;
16     }
17     System.out.println("Tinggi Sadel:"+setSadel);
18 }
19
20
21 }
```

3. buatlah Class dengan nama Sepeda GunungBeraksi.java

```
5 package pewarisan;
6
7 /**
8  *
9  * @author rony
10 */
11 public class SepedaGunungBeraksi {
12     public static void main(String[] args) {
13         // Membuat object
14         SepedaGunung sepedaku = new SepedaGunung();
15         // Memanggil method di object
16         sepedaku.tambahKecepatan(10);
17         sepedaku.ubahGir(2);
18         sepedaku.setSadel(20);
19     }
20 }
```

c. Polymorphism

Polimorfisme merupakan Suatu kemampuan dari sebuah object untuk membolehkan mengambil beberapa bentuk yang berbeda agar tidak terjadi duplikasi object (kemampuan untuk mempunyai beberapa bentuk class yang berbeda). Polimorfisme ini terjadi pada saat suatu object bertipe parent class, akan tetapi pemanggilan constructornya melalui subclass.

Contoh Penerapan

1. Buatlah Sebuah Class dengan Nama Polimorfisme.java

```
class hewan {
    public void suara () {
        System.out.println("Hewan bersuara :");
    }
}
class kuda extends hewan {
    public void suara() {
        System.out.println("Kuda mengikik...");
    }
}
```

```

class kucing extends hewan {
    public void suara () {
        System.out.println("Kucing mengeong...");
    }
}
class ayam extends hewan {
    public void suara() {
        System.out.println("Ayam berkokok...");
    }
}
public class Polimorfisme {
    public static void main(String[] args) {
        hewan HEWAN = new hewan();
        kuda KUDA = new kuda();
        kucing KUCING = new kucing();
        ayam AYAM = new ayam();

        HEWAN.suara();

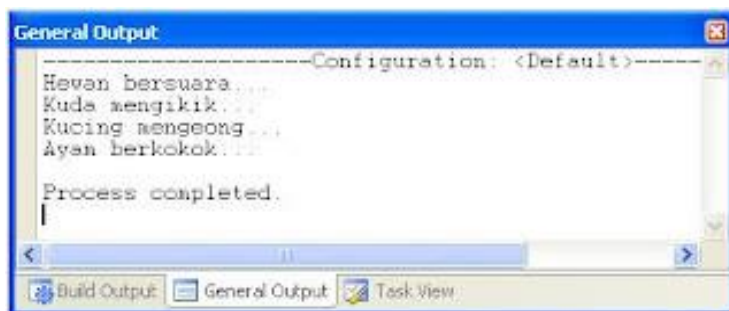
        HEWAN = KUDA;
        HEWAN.suara();

        HEWAN = KUCING;
        HEWAN.suara();

        HEWAN = AYAM;
        HEWAN.suara();
    }
}

```

Output



2.4 Operator Pada Bahasa Java

a. Operator Aritmatika

Operator adalah simbol atau karakter yang biasa dilibatkan dalam program untuk melakukan sesuatu operasi atau manipulasi, seperti penjumlahan, pengurangan dan lain-lain.

a. Unary, yaitu hanya melibatkan sebuah operand pada suatu ekspresi aritmatika.

Contoh : - 7

b. Binary, yaitu melibatkan dua buah operand pada suatu ekspresi aritmatika.

Contoh : 13 + 5

c. Ternary, yaitu melibatkan tiga buah operand pada suatu operasi aritmatika.

Contoh : (8 + 4) * 7 - 5.

Operator	Keterangan	Contoh (x1=4, x2=3)
+	Penjumlahan	$x1 + x2 = 7$
-	Pengurangan	$x1 - x2 = 1$
*	Perkalian	$x1 * x2 = 12$
/	Pembagian	$x1 / x2 = 1,33$
%	Modulus atau sisa pembagian	$x1 \% x2 = 1$

Tabel Operator Aritmatika

Berikut contoh penggunaan operator aritmatika, bisa anda ikuti seperti dibawah ini.

```

1  /**
2   *
3   * @author rony
4   */
5   public class OperatorAritmatika {
6
7   public static void main ( String [] args ) {
8       // Deklarasi dan inisialisasi variabel
9       int i = 9 ;
10      int j = 4 ;
11      // Menampilkan nilai variabel
12      System.out.println ( " Nilai variabel : " ) ;
13      System.out.println ( " i = " + i ) ;
14      System.out.println ( " j = " + j ) ;
15      // Menggunakan operator penjumlahan
16      System.out.println ( " Penjumlahan : " ) ;
17      System.out.println ( " i + j = " + ( i + j ) ) ;
18      // Menggunakan operator pengurangan
19      System.out.println ( " Pengurangan : " ) ;
20      System.out.println ( " i + j = " + ( i - j ) ) ;
21      // Menggunakan operator perkalian
22      System.out.println ( " Perkalian : " ) ;
23      System.out.println ( " i * j = " + ( i * j ) ) ;
24      // Menggunakan operator pembagian
25      System.out.println ( " Pembagian : " ) ;
26      System.out.println ( " i / j = " + ( i / j ) ) ;
27      // Menggunakan operator modulus
28      System.out.println ( " Modulus : " ) ;
29      System.out.println ( " i % j = " + ( i % j ) ) ;
30
31  }
32  }

```

Output yang dihasilkan pada kode program diatas adalah

```

Output - LatihanJava (run)
run:
Nilai variabel :
i = 9
j = 4
Penjumlahan :
i + j = 13
Pengurangan :
i + j = 5
Perkalian :
i * j = 36
Pembagian :
i / j = 2
Modulus :
i % j = 1
BUILD SUCCESSFUL (total time: 3 seconds)

```

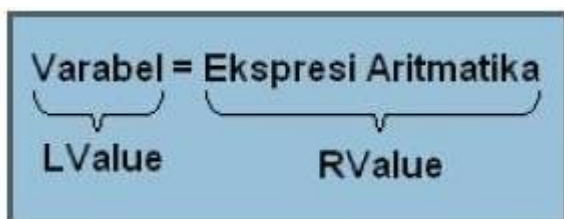
b. Ekspresi aritmatika

Penulisan suatu ekspresi aritmatika pada program java, sangat berkaitan dengan pernyataan pemberi nilai. Karena hasil dari ekspresi aritmatika akan ditampung kedalam suatu variabel.

Bentuk umum penulisan ekspresi aritmatika, seperti dibawah ini :

- a. LValue (Left Value), merupakan berupa variabel tunggal sebagai penampung hasil dari ekspresi Aritmatika
- b. RValue (Right Value), merupakan Ekspresi Aritmatika, bisa berupa unary, binary atau ternary dan variabel lainnya.

Tanda = (sama dengan), dikenal sebagai operator pemberi nilai (Assignment Operator)
Berikut contoh ekspresi aritmatika, bisa anda ikuti seperti dibawah ini.



c. Hierarki Operator Aritmatik

Didalam suatu penulisan ekspresi aritmatika sering kita jumpai menggunakan beberapa operator aritmatika yang berbeda secara bersamaan. Maka dalam prosenya akan berbeda, tergantung dari urutan atau tingkatan operator tersebut. Berikut urutan operator aritmatika, seperti dibawah ini :

Tabel hirarki operator aritmatik

Operator	Penjelasan Operator
*	Ketiga operator ini memiliki tingkatan yang akan diproses lebih dulu. Tingkatan operator sama dan penggunaannya tergantung letak yang yang didepan akan diproses lebih dulu.
/	
%	
+	Kedua operator ini akan diproses kemudian. Tingkatan operator sama dan penggunaannya tergantung letak yang yang didepan akan diproses lebih dulu.
-	

Contoh kasus ekspresi aritmatika, seperti berikut ini :

$A = 8 + 2 * 3 / 6$

Maka langkah-langkah perhitungannya :

Langkah 1:

$A = 2 * 3$ hasilnya 6

$A = 8 + 6 / 6$

Langkah 2:

$A = 6 / 6$ hasilnya 1

$A = 8 + 1$

Langkah 3 :

$A = 9$

Tingkatan operator-operator ini, bisa diabaikan dengan menggunakan tanda kurung buka " ("dan ") ". Jika suatu ekspresi terdapat didalam tanda kurung, maka proses ekspresi tersebut akan diproses terlebih dahulu, tanpa melihat tingkatan operator.

Contoh : $A = (8 + 2) * 3 / 6$

Maka langkah-langkah perhitungannya :

Langkah 1:

$A = 8 + 2$ hasilnya 10

$A = 10 * 3 / 6$

Langkah 2 :

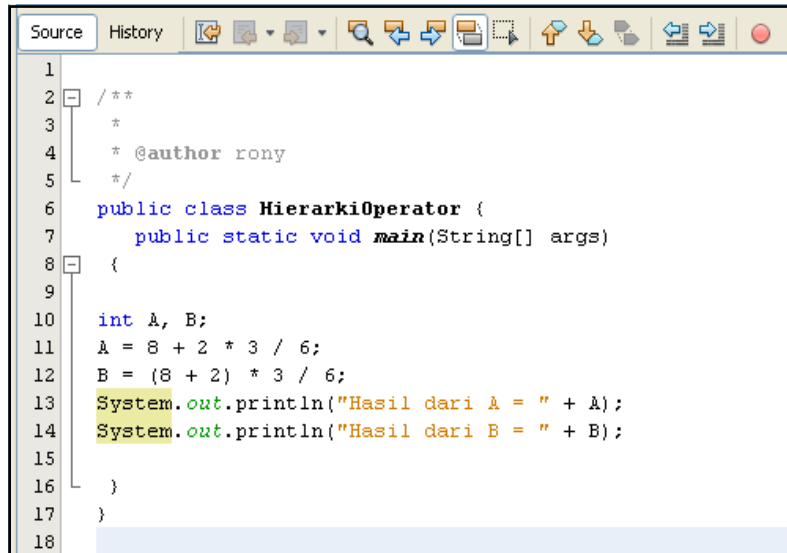
$A = 10 * 3$ hasilnya 30

$A = 30 / 6$

Langkah 3:

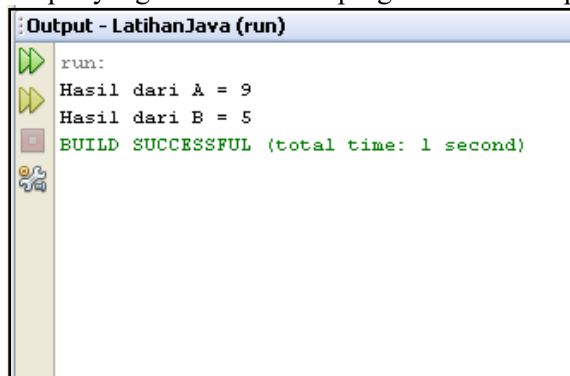
$A=5$

Berikut contoh penggunaan hierarki operator aritmatika, bisa anda ikuti seperti dibawah ini.



```
1
2 /**
3  *
4  * @author rony
5  */
6 public class HierarkiOperator {
7     public static void main(String[] args)
8     {
9
10        int A, B;
11        A = 8 + 2 * 3 / 6;
12        B = (8 + 2) * 3 / 6;
13        System.out.println("Hasil dari A = " + A);
14        System.out.println("Hasil dari B = " + B);
15
16    }
17 }
18
```

Output yang dihasilkan dari program HierarkiOperator.java diatas, seperti dibawah ini :



```
Output - LatihanJava (run)
run:
Hasil dari A = 9
Hasil dari B = 5
BUILD SUCCESSFUL (total time: 1 second)
```

d. Operator Pemberi Nilai

Sebelumnya, kita telah mengenal operator pemberi nilai (Assignment Operator), yaitu menggunakan tanda sama dengan "=", sebagai contoh $A = A + 1$. Dari ekspresi $A = A + 1$, bisa disederhanakan bentuk penulisan ekspresinya, yaitu menjadi $A += 1$.

Notasi $+=$, ini dikenal dengan operator pemberi nilai aritmatika. Java menyediakan beberapa notasi pemberi nilai.

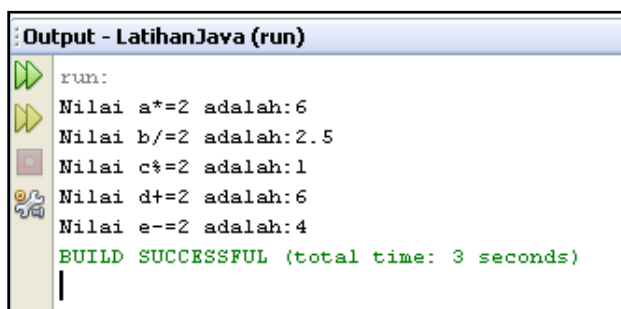
Tabel Operator pemberi nilai

Operasi pada Java	Operator Pemberi Nilai	Contoh Ekspresi	Penggunaan Operator Pemberi Nilai
Perkalian	*=	$A = A * 5$	$A \text{*=} 5$
Pembagian	/=	$A = A / 5$	$A \text{/=} 5$
Sisa Pembagian	$\text{\%}=$	$A = A \% 2$	$A \text{\%}= 2$
Penjumlahan	+=	$A = A + 1$	$A \text{+=} 1$
Pengurangan	-=	$A = A - 4$	$A \text{-=} 4$

Contoh penggunaan Operator Pemberi Nilai

```
1
2  /**
3   *
4   * @author rony
5   */
6  public class OptPemberiNilai {
7      public static void main(String[] args)
8  {
9      int a, c, d, e;
10     double b;
11     a = 3; b = 5; c = 13; d = 4; e = 6;
12
13     //penggunaan assignment operator
14     a *=2;
15     b /=2;
16     c %=2;
17     d +=2;
18     e -=2;
19
20     System.out.println("Nilai a*=2 adalah:"+a);
21     System.out.println("Nilai b/=2 adalah:"+b);
22     System.out.println("Nilai c%=2 adalah:"+c);
23     System.out.println("Nilai d+=2 adalah:"+d);
24     System.out.println("Nilai e-=2 adalah:"+e);
25
26 }
27 }
```

Output dari contoh kode program OptPemberiNilai.java adalah sbb



```
Output - LatihanJava (run)
run:
Nilai a*=2 adalah:6
Nilai b/=2 adalah:2.5
Nilai c%=2 adalah:1
Nilai d+=2 adalah:6
Nilai e-=2 adalah:4
BUILD SUCCESSFUL (total time: 3 seconds)
```

e. Operator Penambah dan Pengurang

Masih berkaitan dengan operator pemberi nilai, Java menyediakan operator penambah dan pengurang, yaitu digunakan untuk menambah satu dan mengurangi satu dari nilai pada dirinya sendiri. Dari contoh penulisan operator pemberi nilai sebagai penyederhanaannya dapat digunakan operator penambah dan pengurang.

Tabel Operator Penambah

Operator	Keterangan
++	Penambahan
--	Pengurangan

Sebagai contoh, terdapat ungkapan aritmatika seperti dibawah ini :

$A = A + 1$ atau $A = A - 1$; maka bentuk ekspresi tersebut bisa disederhanakan menjadi $A += 1$ atau $A -= 1$; hal ini masih dapat disederhanakan menjadi $A++$ atau $A--$. Notasi “++” atau “--” dapat diletakan didepan atau di belakang variabel. Bentuk penulisannya seperti dibawah ini :

A ++ atau ++A dan A — atau —A

Kedua bentuk penulisan notasi ini mempunyai arti yang berbeda. Perbedaan penulisan tersebut, yaitu :

- Jika diletakan didepan variabel, maka proses penambahan atau pengurangan akan dilakukan sesaat sebelum atau langsung pada saat menjumpai ekspresi ini, sehingga nilai variabel tadi akan langsung berubah begitu ekspresi ini ditemukan, sedangkan
- Jika diletakan dibelakang variabel, maka proses penambahan atau pengurangan akan dilakukan setelah ekspresi ini dijumpai atau nilai variabel akan tetap pada saat ekspresi ini ditemukan.

Berikut contoh penggunaan operator penambah dan pengurang, bisa anda ikuti seperti dibawah ini.

```
1
2  /**
3   *
4   * @author rony
5   */
6  public class OptPenambahPengurang {
7      public static void main(String[] args)
8      {
9
10         int a, b;
11         a = 7;
12         b = 3;
13
14         System.out.println("Nilai A = " + a);
15         System.out.println("Nilai ++A = " + ++a);
16         System.out.println("Nilai A++ = " + a++);
17         System.out.println("Nilai A= " + a);
18         System.out.println("-----");
19         System.out.println("Nilai B= " + b);
20         System.out.println("Nilai ++B = " + ++b);
21         System.out.println("Nilai B++ = " + b++);
22         System.out.println("Nilai B= " + b);
23
24     }
25 }
```

Output yang dihasilkan dari penulisan kode program OptPenambahPengurang.java

```
Output - LatihanJava (run)
run:
Nilai A = 7
Nilai ++A = 8
Nilai A++ = 8
Nilai A= 9
-----
Nilai B= 3
Nilai ++B = 4
Nilai B++ = 4
Nilai B= 5
BUILD SUCCESSFUL (total time: 2 seconds)
```

f. Operator Pembanding (Comparison)

Java menyediakan beberapa operator yang digunakan untuk membuat perbandingan-perbandingan antar variabel-variabel, variabel dan literal atau tipe informasi lainnya didalam program.

Operator Pembading (Comparasion) digunakan untuk membandingkan dua buah nilai. Hasil perbandingan operator ini menghasilkan nilai Boolean yaitu True atau False.

Tabel Operator Pembanding:

Operator	Penjelasan
==	Sama dengan
===	Identik dengan (nilai & tipe data sama)
!=	Tidak sama dengan
!==	Tidak identik dengan
>	Lebih besar dari
<	Lebih kecil dari
>=	Lebih besar atau sama
<=	Lebih kecil atau sama

Berikut contoh penggunaan operator pembandingan, bisa anda ikuti seperti dibawah ini.

```

1
2 /**
3  *
4  * @author rony
5  */
6 public class OptPembanding {
7     public static void main(String[] args)
8     {
9         int a, b, c, d, e, f;
10        boolean aa, bb, cc, dd, ee, ff;
11
12        a = 5; b = 6; c = 3; d = 9; e = 8; f = 2;
13
14        aa=a== b;
15        bb=b> c;
16        cc=c< d;
17        dd=d>= e;
18        ee=e<= f;
19        ff=b!= a;
20
21        System.out.println("Apakah"+a+"== "+ b +"? jawabannya " + aa);
22        System.out.println("Apakah"+b+"> "+ c +"? jawabannya " + bb);
23        System.out.println("Apakah"+c+"< "+ d +"? jawabannya " + cc);
24        System.out.println("Apakah"+d+">= "+ e +"? jawabannya " + dd);
25        System.out.println("Apakah"+e+"<= "+ f +"? jawabannya " + ee);
26        System.out.println("Apakah"+b+"!= "+ a +"? jawabannya " + ff);
27    }
28 }

```

Output yang dihasilkan dari program OptPembanding.java diatas, seperti dibawah ini :

```

Output - LatihanJava (run)
run:
Apakah5== 6? jawabannya false
Apakah6> 3? jawabannya true
Apakah3< 9? jawabannya true
Apakah9>= 8? jawabannya true
Apakah8<= 2? jawabannya false
Apakah6!= 5? jawabannya true
BUILD SUCCESSFUL (total time: 2 seconds)

```

g. Operator Logika

Operator Relasi digunakan untuk menghubungkan dua buah operasi relasi menjadi sebuah ungkapan kondisi. Hasil dari operator logika ini menghasilkan nilai boolean True atau False.

Tabel Operator Logika

Operator	Keterangan	Contoh (x1=4, x2=4, y1=2, y2=2)
&&	AND (Apakah kedua kondisi bernilai true semua)	(x1 == x2) && (x1 > y1) (true, karena kondisi pertama (x1 == x2) bernilai true dan kondisi kedua (x1 > y1) juga bernilai true).
	OR (Apakah salah satu kondisi ada yang bernilai true)	(x1 > y2) (x2 == y2) (True, karena ada salah satu kondisi yang bernilai true, yaitu kondisipertama (x1 > y2). Walaupun kondisi kedua bernilai false).
!	Not (Menghasilkan nilai kebalikan dari hasil sebenarnya)	!(x1 > x2) (False, karena hasil awal kondisi tersebut (x1 > x2) bernilai true).

a). Operator Logika AND

Operator logika AND digunakan untuk menghubungkan dua atau lebih ekspresi relasi, akan dianggap BENAR, bila semua ekspresi relasi yang dihubungkan bernilai BENAR.

Contoh :

Ekspresi Relasi-1 $\rightarrow A + 4 < 10$

Ekspresi Relasi-2 $\rightarrow B > A + 5$

Ekspresi Relasi-3 $\rightarrow C - 3 \geq 4$

Penggabungan ketiga ekspresi relasi diatas menjadi ;

$A + 4 < 10 \ \&\& \ B > A + 5 \ \&\& \ C - 3 \geq 4$

Jika nilai $A = 3$; $B = 3$; $C = 7$, maka ketiga ekspresi tersebut mempunyai nilai :

Ekspresi Relasi-1 $\rightarrow A + 4 < 10 \quad 3 + 4 < 10 \rightarrow \text{BENAR}$

Ekspresi Relasi-2 $\rightarrow B > A + 5 \quad 3 > 3 + 5 \rightarrow \text{SALAH}$

Ekspresi Relasi-3 $\rightarrow C - 3 \geq 4 \quad 7 - 3 \geq 4 \rightarrow \text{BENAR}$

Dari ekspresi relasi tersebut mempunyai nilai BENAR, maka

$A + 4 < 10 \ \&\& \ B > A + 5 \ \&\& \ C - 3 \geq 4 \rightarrow \text{SALAH}$

b). Operator Logika OR

Operator logika OR digunakan untuk menghubungkan dua atau lebih ekspresi relasi, akan dianggap BENAR, bila salah satu ekspresi relasi yang dihubungkan bernilai BENAR dan bila semua ekspresi relasi yang dihubungkan bernilai SALAH, maka akan bernilai SALAH.

Contoh :

Ekspresi Relasi-1 $\rightarrow A + 4 < 10$

Ekspresi Relasi-2 $\rightarrow B > A + 5$

Ekspresi Relasi-3 $\rightarrow C - 3 > 4$

Penggabungan ketiga ekspresi relasi diatas menjadi ;

$A + 4 < 10 \ \parallel \ B > A + 5 \ \parallel \ C - 3 > 4$

Jika nilai $A = 3$; $B = 3$; $C = 7$, maka ketiga ekspresi tersebut mempunyai nilai :

Ekspresi Relasi-1 $\rightarrow A + 4 < 10 \rightarrow 3 + 4 < 10 \rightarrow \text{BENAR}$

Ekspresi Relasi-2 $\rightarrow B > A + 5 \rightarrow 3 > 3 + 5 \rightarrow \text{SALAH}$

Ekspresi Relasi-3 $\rightarrow C - 3 > 4 \rightarrow 7 - 3 > 4 \rightarrow \text{SALAH}$

Dilihat ekspresi diatas salah satu ekspresi tersebut mempunyai nilai BENAR, maka ekspresi tersebut tetap bernilai BENAR.

$A + 4 < 10 \ \parallel \ B > A + 5 \ \parallel \ C - 3 > 4 \rightarrow \text{BENAR}$

c). Operator Logika NOT

Operator logika NOT akan memberikan nilai kebalikkan dari ekspresi yang disebutkan. Jika nilai yang disebutkan bernilai BENAR maka akan menghasilkan nilai SALAH, begitu pula sebaliknya.

Contoh :

Ekspresi Relasi $\rightarrow A + 4 < 10$

Penggunaan Operator Logika NOT diatas menjadi : $!(A + 4 < 10)$

Jika nilai $A = 3$; maka ekspresi tersebut mempunyai nilai :

Ekspresi Relasi-1 $\rightarrow A + 4 < 10 \rightarrow 3 + 4 < 10 \rightarrow \text{BENAR}$

Dilihat ekspresi diatas salahsatu ekspresi tersebut mempunyai nilai BENAR dan jika digunakan operator logika NOT, maka ekspresi tersebut akan bernilai SALAH

$!(A + 4 < 10) \rightarrow !(BENAR) = \text{SALAH}$

Contoh Penerapan Operator Logika dalam kode program

```
1
2  /**
3   *
4   * @author rony
5   */
6  public class OptLogika {
7      public static void main(String[] args) {
8
9      System.out.println("Operasi AND");
10     System.out.println("true && true = " + (true && true));
11     System.out.println("true && false = " + (true && false));
12     System.out.println("false && true = " + (false && true));
13     System.out.println("false && false = " + (false && false));
14
15     System.out.println("\nOperasi OR");
16     System.out.println("true || true = " + (true || true));
17     System.out.println("true || false = " + (true || false));
18     System.out.println("false || true = " + (false || true));
19     System.out.println("false || false = " + (false || false));
20
21     System.out.println("\nOperasi XOR");
22     System.out.println("true ^ true = " + (true ^ true));
23     System.out.println("true ^ false = " + (true ^ false));
24     System.out.println("false ^ true = " + (false ^ true));
25     System.out.println("false ^ false = " + (false ^ false));
26
27     System.out.println("\nOperasi NOT");
28     System.out.println("!true = " + (!true));
29     System.out.println("!false = " + (!false));
30 }
31 }
```

Output dari kode program OptLogika.java

```
Output - LatihanJava (run)
run:
Operasi AND
true && true = true
true && false = false
false && true = false
false && false = false

Operasi OR
true || true = true
true || false = true
false || true = true
false || false = false

Operasi XOR
true ^ true = false
true ^ false = true
false ^ true = true
false ^ false = false

Operasi NOT
!true = false
!false = true
BUILD SUCCESSFUL (total time: 0 seconds)
```

BAB III

PERINTAH MASUKAN DAN PENYELEKSIAN PADA PEMROGRAMAN JAVA

3.1. Kopetensi Dasar

Pada pembahasan Bab 3 ini penulis mengajak mendiskusikan mengenai penggunaan Fungsi masukan dan keluaran yang ada pada Bahasa Pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa atau pembaca bisa mendeskripsikan dapat memahami Fungsi masukan, keluaran dan dapat memahami operasi kondisi pada bahasa pemrograman Java. Penulis berharap, diakhir pembahasan, para pembaca bisa :

a. Perintah masukan

- 1). Scanner
- 2). InputStreamReader dan BufferedReader

b. Penyeleleksian

- 1). Penggunaan Pernyataan If, If – Else, Nested If, dan If Majemuk
- 2). Penggunaan Pernyataan Case
- 3). Conditional Operator

3.2. Perintah Masukan

Untuk melakukan perintah masukan, kita akan menggunakan 3 (tiga) cara, yaitu InputStream, BufferedInputStream dan Scanner.

a) Scanner

Class Scanner digunakan secara ekstensif untuk memasukan data dari keyboard. Tidak seperti perintah masukan lainnya, data-data yang dimasukan misalnya berupa angka, tidak perlu dilakukan konversi dari string ke integer atau tipe data lainnya. Untuk menggunakan Class Scanner, harus menyertakan package java.util. Untuk membaca baris dari text yang diinputkan dengan menggunakan metoda next(). Supaya string yang dibaca utuh termasuk spasi, dengan menggunakan nextLine(). Untuk membaca data berupa nilai integer atau tipe data angka lainnya, sebagai contoh bisa menggunakan metoda,

seperti dibawah ini :

nextInt() untuk membaca nilai integer

nextDouble() untuk membaca nilai double.

nextFloat() untuk membaca nilai float.

Contoh pennggunaan fungsi masukan scanner pada LatScanner.java

```
3 import java.util.Scanner;
4 /**
5  *
6  * @author rony
7  */
8 public class LatScanner {
9     public static void main(String args[])
10    {
11        //membuat objek baru
12        Scanner input = new Scanner(System.in);
13
14        //deklarasi variabel
15
16        String nama;
17        int n2;
18        double n1, n3;
19
20        System.out.print("Masukan Nama Anda : ");
21        nama = input.nextLine();
22        System.out.print("Masukan Nilai 1 : ");
23        n1 = input.nextDouble();
24        System.out.print("Masukan Nilai 2 : ");
25        n2 = input.nextInt();
26
27        n3 = n1 + n2;
28        System.out.println("\nNama Anda : " + nama);
29
30        System.out.println("Nilai Anda : " + n3);
31    }
32 }
```

Berikut penggunaan Scanner dan pembuatan Class yang digunakan di dalam main Class, untuk masukan data dari keyboard.

1. buatlah sebuah Class dengan nama ClassScanner.java

```
2 import java.util.Scanner;
3 /**
4  *
5  * @author rony
6  */
7 public class ClassScanner {
8     public String nama;
9     public double n1;
10    public int n2,n3;
11    Scanner input = new Scanner(System.in);
12
13    public String getnama(){
14        return nama;
15    }
16
17    public void inputScanner(){
18
19        System.out.print("Masukan Nama Anda : ");
20        nama = input.nextLine();
21        System.out.print("Masukan Nilai 1 : ");
22        n1 = input.nextDouble();
23        System.out.print("Masukan Nilai 2 : ");
24        n2 = input.nextInt();
25    }
26
27    public double rata(){
28
29        return ((n1 + n2)/2);
30    }
31 }
```

Kode program ClassScanner yang telah di buat diatas adalah sebuah class yang kan digunakan didalam main Class sesuai dengan konsep dari pemrograman berorientasi Objek kemudian buatlah sebuah main class dengan nama Aksi Scanner

```
1 import java.util.Scanner;
2 /**
3  *
4  * @author rony
5  */
6 public class AksiScanner {
7     public static void main(String args[])
8     {
9         //membuat objek baru
10        ClassScanner scan=new ClassScanner();
11
12        scan.inputScanner();
13        System.out.println("\nNama Anda : " + scan.getnama());
14        System.out.println("Nilai Anda : " + scan.rata());
15    }
16 }
```

Sedangkan pada kode program AksiProgram.Java merupakan sebuah main class yang dalam penulisan kode programnya hanya perlu memanggil method yang telah dibuat di dalam ClassScanner Berikut output dari kode program AksiScanner.java

```
Output - LatihanJava (run)
run:
Masukan Nama Anda : Rony
Masukan Nilai 1 : 90
Masukan Nilai 2 : 80
Nama Anda : Rony
Nilai Anda : 85.0
BUILD SUCCESSFUL (total time: 11 seconds)
```

Latihan

1. Buatlah Class dengan nama LatScanner.java sebagai class dan LatScannerAksi.java sebagai main Class
2. Di dalam LatScanner.java buatlah metod untuk penginputan dan juga proses untuk mencari nilai rata-rata sesuai dengan ketentuan
3. Tampilan Input

Input :
Masukan NIM :
Masukan Nama :
Masukan Nilai Absen :
Masukan Nilai Tugas :
Masukan Nilai UTS :
Masukan Nilai UAS :
Proses :
$\text{Rata} = (10\% * na) + (20\% * nt) + (30\% * nuts) + (40\% * nuas)$

4. Tampilan Output

PROGRAM NILAI MAHASISWA	

NIM	:
NAma Mahasiswa	:
Nilai Absen	:
Nilai TUGas	:
Nilai UTS	:
Nilai UAS	:

Rata-Rata	:

b). BufferedReader

InputStreamReader digunakan membaca arus byte stream dan mengkonversi byte-byte ke dalam nilai-nilai bilangan bulat yang merepresentasikan karakter-karakter Unicode.

Kelas BufferedReader membaca masukan Stream karakter dan penyangga tersebut untuk efisiensi. Pada penggunaannya harus mempunyai Reader Object untuk membuat versi buffered. Berikut konstrutor yang digunakan untuk membuat BufferedReader.

- `BufferedReader(Reader)` digunakan untuk membuat penyangga karakter stream yang berhubungan dengan Reader Objek yang ditetapkan.
- `BufferedReader (Reader, int)` digunakan untuk membuat penyangga karakter stream yang berhubungan dengan Reader Objek yang ditetapkan dan dengan penyangga ukuran integer.

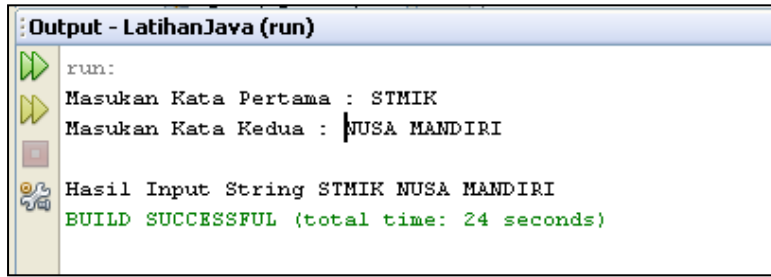
Penyangga Karakter Stream dapat dibaca menggunakan metoda `read()` dan `read(char[], int, int)` untuk menguraikan `FileReader`, serta dapat juga membaca baris dari teks dengan menggunakan `readLine()`. metoda `readLine()`, digunakan untuk kembalikan String objek yang berisi next line dari teks pada stream, tidak termasuk karakter atau karakter-karakter yang merepresentasikan end-of-line. Jika akhir stream dicapai, maka nilai string yang ingin dikembalikan bernilai dengan null. end-of-line ditandai dengan beberapa pernyataan berikut :

- newline karakter (`'\n'`)
- carriage return character (`'\r'`)
- carriage return character yang diikuti oleh satu newline (`"\n\r"`)
-

Berikut kode Program penggunaan `InputStreamReader` dan `Buffered`, untuk masukan data dari keyboard.

```
1 import java.io.*;
2
3 /**
4  *
5  * @author rony
6  */
7
8 public class LatBuffered {
9     public static void main(String args[]) throws Exception
10    {
11        //membuat objek baru
12        InputStreamReader keyreader = new InputStreamReader(System.in);
13        BufferedReader input = new BufferedReader(keyreader);
14        //deklarasi variabel
15        String kata1, kata2;
16
17        System.out.print("Masukan Kata Pertama : ");
18        kata1 = input.readLine();
19        System.out.print("Masukan Kata Kedua : ");
20        kata2 = input.readLine();
21        System.out.println("\nHasil Input String " + kata1 + " " + kata2);
22    }
23 }
24
25 }
```

Tampilan output



```
run:
Masukan Kata Pertama : STMIK
Masukan Kata Kedua : NUSA MANDIRI
Hasil Input String STMIK NUSA MANDIRI
BUILD SUCCESSFUL (total time: 24 seconds)
```

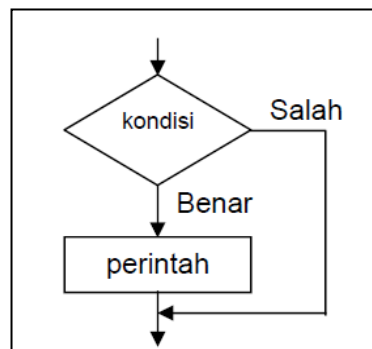
Latihan...!!!

Terjemahkanlah contoh program diatas dengan menggunakan konsep pembuatan class dan main class
Buatlah class input sebagai class terpisah yang akan digunakan di dalam class utama yang hanya menampilkan proses output.

3.3. Operator Penyeleksian Pada Pemrograman Bahasa Java

3.3.1. Pernyataan IF

Pernyataan if mempunyai pengertian, “ Jika kondisi bernilai benar, maka perintah akan dikerjakan dan jika tidak memenuhi syarat maka akan diabaikan”. Dari pengertian tersebut dapat dilihat dari diagram alir berikut:



Flowchart pernyataan If

Penulisan kondisi harus didalam tanda kurung dan berupa ekspresi relasi dan penulisan pernyataan dapat berupa sebuah pernyataan tunggal, pernyataan majemuk atau pernyataan kosong. Jika pemakaian if diikuti dengan pernyataan majemuk, bentuk penulisannya sebagai berikut :

```
if (kondisi)
    pernyataan;
```

Jika lebih dari satu pernyataan hasu diapit dengan tanda kurung kurawal

```
if (kondisi)
{
    pernyataan;
    .....
}
```

Menentukan besarnya potongan dari pembelian barang yang diberikan seorang pembeli, dengan kriteria :

- Tidak ada potongan jika total pembelian kurang dari Rp. 50.000,-
- Jika total pembelian lebih dari atau sama dengan Rp. 50.000,- potongan yang diterima sebesar 20% dari total pembelian.

Berikut contoh penggunaan pernyataan if sederhana dalam bentuk program terseruktur , bisa anda ikuti seperti dibawah ini.

```
import java.util.*;

/**
 *
 * @author rony
 */
public class LatIf
{
    public static void main(String[] args)
    {
        double tot_beli, potongan=0, jum_bayar=0;
        Scanner input = new Scanner(System.in);

        System.out.print("Total Pembelian Rp. ");
        tot_beli = input.nextDouble();
        if (tot_beli >= 50000)
            potongan = 0.2 * tot_beli;
        System.out.println("Besarnya Potongan Rp. " + potongan);
        jum_bayar = tot_beli - potongan;
        System.out.println("Jumlah yang harus dibayarkan Rp. " + jum_bayar);
    }
}
```

Output yang dihasilkan dari program LatIf.java diatas, seperti dibawah ini :

```
Output - LatihanJava (run)

run:
Total Pembelian Rp. 85000
Besarnya Potongan Rp. 17000.0
Jumlah yang harus dibayarkan Rp. 68000.0
BUILD SUCCESSFUL (total time: 8 seconds)
```

Sekrang kita pahami lagi konsep OOP dengan memisahkan kode program menjadi beberapa class dengan method yang ada pada class yang di buat.

Buatlah sebuah class dengan nama IfClass.java dengan kode program seperti di bawah ini

```
package KondisiIf;

/**
 *
 * @author rony
 */
public class IfClass {
    public double TotBeli, potongan;

    public void setTotalBeli(double a) {
        TotBeli=a;
    }

    public double getPotongan() {
        if (TotBeli >= 50000) {
            potongan = 0.2 * TotBeli;
        }
        return potongan;
    }

    public double JumlahBayar()
    {
        return (TotBeli-potongan);
    }
}
```


Kemudian buatlah sebuah class sebagai class utama dengan nama IfClassAksi.java dengan kode program di bawah ini

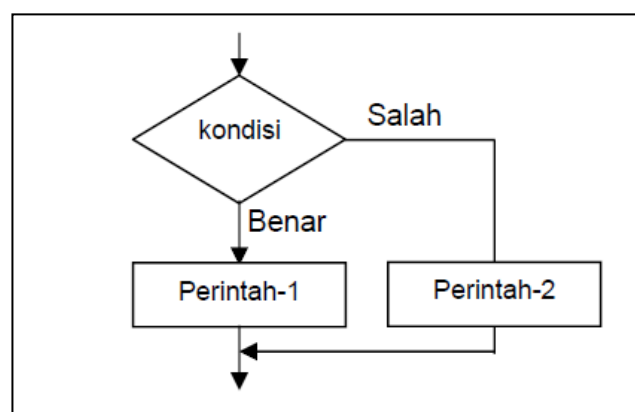
```
1
2 package KondisiIf;
3
4 import java.util.Scanner;
5
6 /**
7  *
8  * @author rony
9  */
10 public class IfClassAksi {
11     public static void main(String[] args)
12     {
13         double tot beli;
14         Scanner input = new Scanner(System.in);
15         IfClass fungsiif=new IfClass();
16
17         System.out.print("Total Pembelian Rp. ");
18         fungsiif.TotBeli = input.nextDouble();
19
20         System.out.println("Besarnya Potongan Rp. " + fungsiif.getPotongan());
21         System.out.println("Jumlah yang harus dibayarkan Rp. " +fungsiif.JumlahBayar());
22     }
23 }
```

Lalu jalankan file diatas dan akan tampil output seperti di bawah ini

```
Output - LatihanJava (run)
run:
Total Pembelian Rp. 85000
Besarnya Potongan Rp. 17000.0
Jumlah yang harus dibayarkan Rp. 68000.0
BUILD SUCCESSFUL (total time: 11 seconds)
```

3.3.2. Pernyataan IF-ELSE

Pernyataan if-else mempunyai pengertian, “ Jika kondisi bernilai benar, maka perintah-1 akan dikerjakan dan jika tidak memenuhi syarat maka akan mengerjakan perintah-2”. Dari pengertian tersebut dapat dilihat dari diagram alir berikut :



Flowchart proses If-Else

Perintah-1 dan perintah-2 dapat berupa sebuah pernyataan tunggal, pernyataan majemuk atau pernyataan kosong. Jika pemakaian if-else diikuti dengan pernyataan majemuk, bentuk penulisannya sebagai berikut:

```
if (kondisi)
    pernyataan-1;
else
    pernyataan-1;
```

Jika lebih dari satu pernyataan harus diapit dengan tanda kurung kurawal.

```
if (kondisi)
{
    perintah-1;
    ...
}
else
{
    perintah-2;
    ...
}
```

Berikut contoh penggunaan pernyataan if – else dalam bentuk program terstruktur, bisa anda ikuti seperti di bawah ini.

```
3
4 import java.util.Scanner;
5
6 /**
7  *
8  * @author rony
9  */
10 public class LatIfElse {
11     public static void main(String[] args) {
12         Scanner input = new Scanner(System.in);
13         String keterangan,nama;
14         int nilai;
15
16         System.out.print("Masukan Nama Siswa:");
17         nama = input.nextLine();
18         System.out.print("Masukan Nilai Akhir :");
19         nilai = input.nextInt();
20         if(nilai > 70){
21             // perintah jika kondisi true
22             keterangan="lulus";
23         }else{
24             // perintah jika kondisi false
25             keterangan="Gagal";
26         }
27         System.out.println("Hasil Akhir");
28         System.out.println("=====");
29         System.out.println("Nama Siswa           :"+nama);
30         System.out.println("Nilai Akhir Yang di dapat :"+nilai);
31         System.out.println("Keterangan           :"+keterangan);
32         System.out.println("=====");
33     }
34 }
```

Tampilan output kode program LatIfElse.java

```
Output - LatihanJava (run)
run:
Masukan Nama Siswa:Razita
Masukan Nilai Akhir :90
Hasil Akhir
=====
Nama Siswa           :Razita
Nilai Akhir Yang di dapat :90
Keterangan           :lulus
=====
BUILD SUCCESSFUL (total time: 12 seconds)
```

Sekarang mari kita rubah kode program terstruktur diatas dengan konsep class dan main class. Buatlah sebuah class dengan nama IfElseClass.java dengan kode program sebagai berikut

```
1
2
3 import java.util.Scanner;
4
5 /**
6  *
7  * @author rony
8  */
9 public class IfElseClass {
10     public String nama,ket;
11     public int nilAkhir;
12     Scanner input = new Scanner(System.in);
13
14     public void setInputData(){
15         System.out.print("Masukan Nama Siswa:");
16         nama = input.nextLine();
17         System.out.print("Masukan Nilai Akhir :");
18         nilAkhir = input.nextInt();
19     }
20
21     public String getKeterangan(){
22         if(nilAkhir > 70){
23             // perintah jika kondisi true
24             ket="lulus";
25         }else{
26             // perintah jika kondisi false
27             ket="Gagal";
28         }
29         return ket;
30     }
31 }
32
```

Setelah IfElseClass di buat maka kita akan membuat main class dengan nama IfElseAksi.java dengan kode program sebagai berikut.

```

3
4  /**
5   *
6   * @author rony
7   */
8  public class IfElseAksi {
9      public static void main(String[] args) {
10
11          //pembuatan objek baru dari class IfElseClass.java
12          IfElseClass ifclass=new IfElseClass();
13
14          ifclass.setInputData();
15          ifclass.getKeterangan();
16          System.out.println("Hasil Akhir");
17          System.out.println("=====");
18          System.out.println("Nama Siswa           :"+ifclass.nama);
19          System.out.println("Nilai Akhir Yang di dapat :"+ifclass.nilaiAkhir);
20          System.out.println("Keterangan           :"+ifclass.getKeterangan());
21          System.out.println("=====");
22      }
23  }
24

```

Kemudian jalankan kode program IfElseAksi.java dan tampilan outputnya akan tampil sebagai berikut:

```

Output - LatihanJava (run)
run:
Masukan Nama Siswa:Rizka
Masukan Nilai Akhir :85
Hasil Akhir
=====
Nama Siswa           :Rizka
Nilai Akhir Yang di dapat :85
Keterangan           :lulus
=====
BUILD SUCCESSFUL (total time: 11 seconds)

```

Mudah kan sekarang kita coba latihan dengan konsep OOP.

1. Buat sebuah class dengan nama latIfClass.java sebagai class yang terdiri dari method untuk menentukan penyeleksian kondisi untuk menentukan nama kua dan harga dan juga method untuk menentukan Total bayar.
2. Kemudian buatlah sebuah class sebagai main class dengan nama LatIfAksi.java untuk menyelesaikan kasus seperti di bawah ini.

Tampilan Input.

Masukan nama pembeli	=
Masukan kode[B001 C001 T001]	=
Masukan jumlah beli	=

Tampilan Output dan ketentuan

Proses :		
Kode kue	nama kue	harga
B001	Bolu	10000
C001	Cake	20000
T001	Tart	15000
Total : harga x jumlah		
Output :		
PENJUALAN KUE PADA TOKO KUE 'LEZAT'		
JL. Sagu No.27A Kebagusan , Jakarta Selatan		

PEMBELI	:	
KODE KUE	:	
NAMA KUE	:	
HARGA	:RP.	
JUMLAH BELI	:	

TOTAL	:RP.	

Selamat Mengerjakan.

3.3.3.. IF Bersarang Nested-If

Nested if merupakan pernyataan if berada didalam pernyataan if yang lainnya. Bentuk penulisan pernyataan Nested if adalah :

```
if(syarat)
if(syarat)
... perintah;
else
... perintah;
else
if(syarat)
... perintah;
else
... perintah;
```

Contoh :

Suatu perusahaan memberikan komisi kepada para salesman dengan ketentuan sebagai berikut:

- Bila salesman dapat menjual barang hingga Rp. 20.000 ,- , akan diberikan uang jasa sebesar Rp. 10.000 ditambah dengan uang komisi Rp. 10% dari pendapatan yang diperoleh hari itu.
- Bila salesman dapat menjual barang diatas Rp. 20.000 ,- , akan diberikan uang jasa sebesar Rp. 20.000 ditambah dengan uang komisi Rp. 15% dari pendapatan yang diperoleh hari itu.
- Bila salesman dapat menjual barang diatas Rp. 50.000 ,- , akan diberikan uang jasa sebesar Rp. 30.000 ditambah dengan uang komisi Rp. 20% dari pendapatan yang diperoleh hari itu.

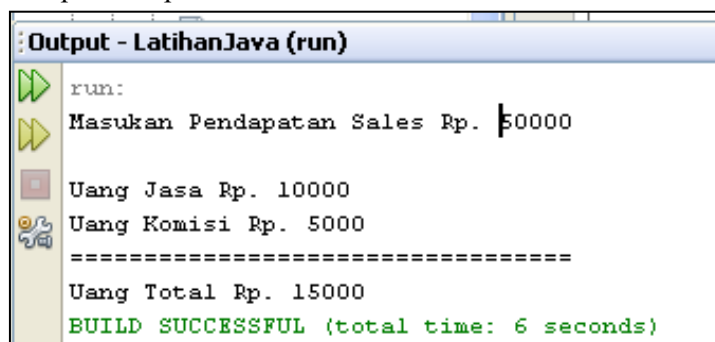
Berikut contoh penggunaan pernyataan nested - if, bisa anda ikuti seperti dibawah ini.

```

8 public class NetsedIf {
9     public static void main(String[] args) {
10         int pendapatan;
11         double jasa, komisi, total;
12         //deklarasi objek
13         Scanner input = new Scanner(System.in);
14
15         System.out.print("Masukan Pendapatan Sales Rp. ");
16         pendapatan = input.nextInt();
17
18         if (pendapatan >= 0 && pendapatan <= 200000) {
19             jasa=10000;
20             komisi=0.1*pendapatan;
21         }
22         else
23         if(pendapatan<=500000) {
24             jasa=20000;
25             komisi=0.15*pendapatan;
26         }
27         else{
28             jasa=30000;
29             komisi=0.2*pendapatan;
30         }
31         /* menghitung total */
32         total = komisi+jasa;
33         System.out.println("\nUang Jasa Rp. " + (int) jasa);
34         System.out.println("Uang Komisi Rp. " + (int) komisi);
35         System.out.println("=====");
36         System.out.println("Uang Total Rp. " + (int) total);
37     }
38 }

```

Tampilan output :



```

Output - LatihanJava (run)
run:
Masukan Pendapatan Sales Rp. 50000
Uang Jasa Rp. 10000
Uang Komisi Rp. 5000
=====
Uang Total Rp. 15000
BUILD SUCCESSFUL (total time: 6 seconds)

```

Catatan

Jika telah memahami konsep OOP cobalah buat contoh yang telah dibuat diatas dengan konsep tersebut.

Atau buatlah kasus yang berbedanya tetap pada konsep OOP.

Perbanyaklah focus latihan penyeleksian kondisi dengan konsep OOP.

3.3.4. If-Else Majemuk

Bentuk dari if-else bertingkat sebenarnya mirip dengan nested if, keuntungan penggunaan if- else bertingkat dibanding dengan nested if adalah penggunaan bentuk penulisan yang lebih sederhana.

Bentuk Penulisannya

if (syarat)

```

{
... perintah;
... perintah;
}

```

else if (syarat)

```

{
... perintah;
... perintah;
}
else
{
... perintah;
... perintah;
}

```

Contoh :

Suatu perusahaan memberikan komisi kepada para salesman dengan ketentuan sebagai berikut:

- Bila salesman dapat menjual barang hingga Rp. 200.000 ,- , akan diberikan uang jasa sebesar Rp. 10.000 ditambah dengan uang komisi Rp. 10% dari pendapatan yang diperoleh hari itu.
- Bila salesman dapat menjual barang diatas Rp. 200.000 ,- , akan diberikan uang jasa sebesar Rp. 20.000 ditambah dengan uang komisi Rp. 15% dari pendapatan yang diperoleh hari itu.
- Bila salesman dapat menjual barang diatas Rp. 500.000 ,- , akan diberikan uang jasa sebesar Rp. 30.000 ditambah dengan uang komisi Rp. 20% dari pendapatan yang diperoleh hari itu.

Berikut contoh penggunaan pernyataan if – else majemuk, bisa anda ikuti seperti dibawah ini.

```

3 import java.util.Scanner;
4 /**
5  *
6  * @author rony
7  */
8 public class IfElseMajemuk {
9     public static void main(String[] args){
10         int pendapatan;
11         double jasa, komisi, total;
12         //deklarasi objek
13         Scanner input = new Scanner(System.in);
14         System.out.print("Masukan Pendapatan Sales Rp. ");
15         pendapatan = input.nextInt();
16
17         if (pendapatan >= 0 && pendapatan <= 200000) {
18             jasa=10000;
19             komisi=0.1*pendapatan;
20         }
21         else if(pendapatan<=500000) {
22             jasa=20000;
23             komisi=0.15*pendapatan;
24         }
25         else {
26             jasa=30000;
27             komisi=0.2*pendapatan;
28         }
29         /* menghitung total */
30         total = komisi+jasa;
31         System.out.println("\nUang JasaRp. " + (int) jasa);
32         System.out.println("Uang Komisi Rp. " + (int) komisi);
33         System.out.println("=====");
34         System.out.println("Uang Total Rp. " + (int) total);
35     }
36 }

```

Jalankan kode program diatas dan akan tampil output seperti di bawah ini.

```

Output - LatihanJava (run)
run:
Masukan Pendapatan Sales Rp. 75000
Uang JasaRp. 10000
Uang Komisi Rp. 7500
=====
Uang Total Rp. 17500
BUILD SUCCESSFUL (total time: 9 seconds)

```

Catatan

Untuk lebih memahami konsep OOP cobalah buat contoh yang telah dibuat diatas dengan konsep tersebut. Atau buatlah kasus yang berbeda dengan tetap pada konsep OOP.

Perbanyaklah focus latihan penyeleksian kondisi dengan konsep OOP.

3.3.4. Pernyataan switch - case

Bentuk dari switch - case merupakan pernyataan yang dirancang khusus untuk menangani pengambilan keputusan yang melibatkan sejumlah atau banyak alternatif. Pernyataan switch -case ini memiliki kegunaan sama seperti if – else bertingkat, tetapi penggunaannya hanya untuk memeriksa data yang bertipe primitif integer saja.

Bentuk penulisan perintah ini sebagai berikut :

```
switch (ekspresi integer)
{
case konstanta-1 :
... perintah;
... perintah;
break;
case konstanta-2 :
... perintah;
... perintah;
break;
.....
.....
default :
... perintah;
... perintah;
}
```

Setiap cabang akan dijalankan jika syarat nilai konstanta tersebut dipenuhi dan default akan dijalankan jika semua cabang diatasnya tidak terpenuhi.

Pernyataan break menunjukan bahwa perintah siap keluar dari switch. Jika pernyataan ini tidak ada, maka program akan diteruskan ke cabang – cabang yang lainnya.

Berikut contoh penggunaan pernyataan switch - case, bisa anda ikuti seperti dibawah ini.


```

Contoh :
import java.util.*;
class LatScan4
{
public static void main(String[]args)
{
int kd;
String nb="";
Scanner input=new Scanner(System.in);
System.out.print("masukan kode barang[1..3]:");
kd=input.nextInt();

switch(kd)
{
    case 1:
        nb="Alat Olah raga";
        break;
    case 2:
        nb="Alat Elektronik";
        break;
    case 3:
        nb="Alat Alat Masak";
        break;
    default:
        nb=" Anda Salah Kode !!!";
        break;
}
System.out.println("\nNama Barang : " +nb);
}

```

Latihan..!!

DINGIN DAMAI, memberikan Honor tetap kepada karyawan kontraknya sebesar Rp. 300,000,- per bulan, dengan memperoleh tunjangan-tunjangan sebagai berikut :

a. Tunjangan Jabatan

Golongan	Persentase
1	5%
2	10%
3	15%

Sebagai contoh : Jika seorang karyawan tersebut dengan golongan 3, maka mendapatkan tunjangan sebesar $15\% \times \text{Rp. } 300,000,-$

b. Tunjangan Pendidikan

Kode	Pendidikan	Persentase
1	SMU	2,5%
2	D3	5%
3	S1	7,5%

c. Honor Lembur

Jumlah jam kerja normal sebanyak 8 Jam Kerja. Honor lembur diberikan jika jumlah jam kerja lebih dari 8 jam, maka kelebihan jam kerja tersebut dikalikan dengan honor lembur perjam sebesar Rp. 2,500 untuk setiap kelebihan jam kerja perharinya.

d. Tampilan yang diinginkan sebagai berikut :

Layar Masukkan

Program Hitung Honor Karyawan Kontrak

PT. DINGIN DAMAI

Nama Karyawan : ... <di input>

Golongan : ... <di input>

Pendidikan (SMU/D3/S1) : ... <di input>

Jumlah Jam Kerja : ... <di input>

Layar Keluaran

Karyawan yang bernama : ... <tampil data>

Honor yang diterima

Honor Tetap Rp. <hasil proses>

Tunjangan jabatan Rp. <hasil proses>

Tunjangan Pendidikan Rp. <hasil proses>

Honor Lembur Rp. <hasil proses>

=====+

Honor Yang Diterima Rp. <hasil proses>

e. kerjakan latihan diatas dengan menggunakan konsep class dan main class(OOP)

f. buatlah class latihanIfclass.java yang didalamnya terdapat method untuk penyeleksian kondis golongan da pendidikan serta method perhitungan honor yang di terima.

BAB IV

PERULANGAN PEMORGRAMAN JAVA

4.1. Kopetensi Dasar

Pada pembahasan Bab 4 ini penulis mengajak mendiskusikan mengenai penggunaan operasi kondisi dan array yang pada Bahasa Pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa atau pembaca bisa mendeskripsikan dapat memahami perintah perulangan dan array pada bahasa pemrograman Java. Penulis berharap, diakhir pembahasan, para pembaca bisa :

- a. Penggunaan Pernyataan Perulangan for, nested for dan perulangan tak berhingga dengan perulangan for.
- b. Penggunaan Pernyataan Perulangan While dan Do While
- c. Perintah Break dan Continue
- d. Penggunaan pendeklarasian array berdimensi satu, memasukan nilai ke array dimensi satu dan mengambil nilai dari array dimensi satu.
- e. Penggunaan pendeklarasian array berdimensi dua, memasukan nilai ke array dimensi dua dan mengambil nilai dari array dimensi dua
- f. Penggunaan pendeklarasian array berdimensi tiga, memasukan nilai ke array dimensi tiga dan mengambil nilai dari array dimensi tiga.

4.2. Pernyataan for

Perulangan yang pertama adalah for. Bentuk umum pernyataan for sebagai berikut :

for (inisialisasi; syarat pengulangan; pengubah nilai pencacah)

Bila pernyataan didalam for lebih dari satu maka pernyataan-pernyataan tersebut harus diletakan didalam tanda kurung.

```
for ( inisialisasi; syarat pengulangan; pengubah nilai pencacah )
{
    pernyataan / perintah;

    pernyataan / perintah;
    pernyataan / perintah;
}
```

Kegunaan dari masing-masing argumen for diatas adalah :

- Inisialisasi : merupakan bagian untuk memberikan nilai awal untuk variabel-variabel tertentu.
- Syarat Pengulangan : memegang kontrol terhadap pengulangan, karena bagian ini yang akan menentukan suatu perulangan diteruskan atau dihentikan.
- Pengubah Nilai Pencacah : mengatur kenaikan atau penurunan nilai pencacah.

Contoh :

Sebagai contoh program untuk mencetak bilangan dari 1 hingga 10 secara menaik, secara selengkapnya seperti dibawah ini:

```
3
4  /**
5   *
6   * @author rony
7   */
8   public class contohFor {
9       public static void main(String[] args)
10      {
11          int a;
12          for(a = 1; a <= 10; ++a)
13              System.out.println(a);
14      }
15  }
```

Mencetak Bilangan ganjil

```
3
4  /**
5   *
6   * @author rony
7   */
8  public class ForGanjil {
9      public static void main(String[] args)
10     {
11         int a;
12         for(a = 1; a <= 10; a+=2)
13             System.out.println(a);
14     }
15 }
16
```

4.3. Pernyataan nested - for

Pernyataan Nested for adalah suatu perulangan for didalam perulangan for yang lainnya. Bentuk umum pernyataan Nested for sebagai berikut :

```
for ( inisialisasi; syarat pengulangan; pengubah nilai pencacah )
{
    for ( inisialisasi; syarat pengulangan; pengubah nilai pencacah )
    {
        pernyataan / perintah;
    }
}
```

Didalam penggunaan nested-for, perulangan yang didalam terlebih dahulu dihitung hingga selesai, kemudian perulangan yang diluar diselesaikan.

Sebagai contoh program menerapkan perintah nested - for, selengkapnya seperti dibawah ini:

```
6
7  /**
8   *
9   * @author rony
10  */
11 public class NestedFor {
12     public static void main(String[] args)
13     {
14         int a, b;
15
16         for(a = 1; a <= 5; a++)
17         {
18             System.out.println();
19             for(b = a; b <= 5; b++)
20             {
21                 System.out.print(a);
22             }
23         }
24         System.out.println("\n");
25     }
26 }
```

Tampilan Output

```
Output - LatihanJava (run)
run:
11111
22222
333
44
5

BUILD SUCCESSFUL (total time: 2 seconds)
```

Dapat anda analisa kenapa bisa hasilnya menjadi seperti gambar 5.6, diatas ?, berikut dapat dilihat pada tabel proses berikut ini :

a	a<=5	? \n	b=a	b<=5	? a	b++	a++
1	1<=5	✓	1	1<=5	1	2	
			2	2<=5	1	3	
			3	3<=5	1	4	
			4	4<=5	1	5	
			5	5<=5	1	6	
			6	6<=5	-	-	2
2	2<=5	✓	2	2<=5	2	3	
			3	3<=5	2	4	
			4	4<=5	2	5	
			5	5<=5	2	6	
			6	6<=5	-	-	3
3	3<=5	✓	3	3<=5	3	4	
			4	4<=5	3	5	
			5	5<=5	3	6	
			6	6<=5	-	-	4
4	4<=5	✓	4	4<=5	4	5	
			5	5<=5	4	6	
			6	6<=5	-	-	5
5	5<=5	✓	5	5<=5	5	6	
	6<=5	Proses perulangan Selesai					

4.4. Pernyataan while

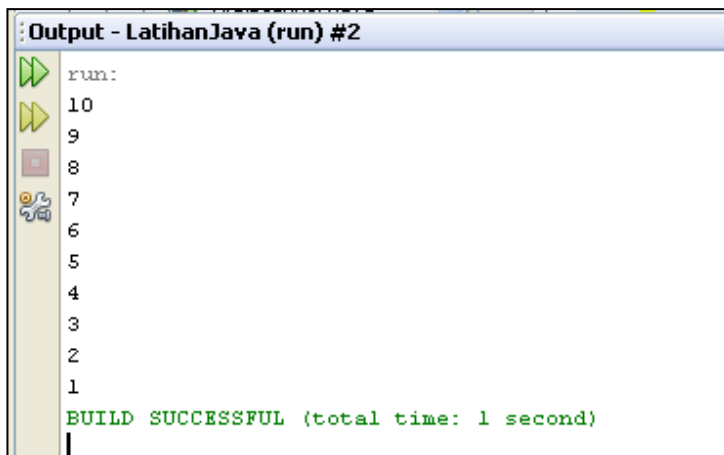
Pernyataan perulangan while merupakan instruksi perulangan yang mirip dengan perulangan for. Bentuk perulangan while dikendalikan oleh syarat tertentu, yaitu perulangan akan terus dilaksanakan selama syarat tersebut terpenuhi. Bentuk umum perulangan while, sebagai berikut :

```
while ( syarat )
Pernyataan / perintah ;
Bentuk umum perulangan while, dengan lebih dari perintah / pernyataan, sebagai berikut :
while ( syarat )
{
Pernyataan / perintah ;
Pernyataan / perintah ;
}
```

Sebagai contoh program menerapkan perintah perulangan while, selengkapnya seperti dibawah ini:

```
7  /**
8   *
9   * @author rony
10  */
11  public class ContohWhile {
12      public static void main(String[] args)
13      {
14          int bil=10;
15          while(bil>=1)
16          {
17              System.out.println(bil);
18              --bil;
19          }
20      }
21  }
```

Output yang dihasilkan dari program ContohWhile.java diatas, seperti dibawah ini :



```
run:
10
9
8
7
6
5
4
3
2
1
BUILD SUCCESSFUL (total time: 1 second)
```

4.5. Pernyataan do-While

Pernyataan perulangan do - while merupakan bentuk perulangan yang melaksanakan perulangan terlebih dahulu dan pengujian perulangan dilakukan dibelakang. Bentuk umum perulangan do - while, sebagai berikut :

```
do
pernyataan / perintah ;
while ( syarat );
```

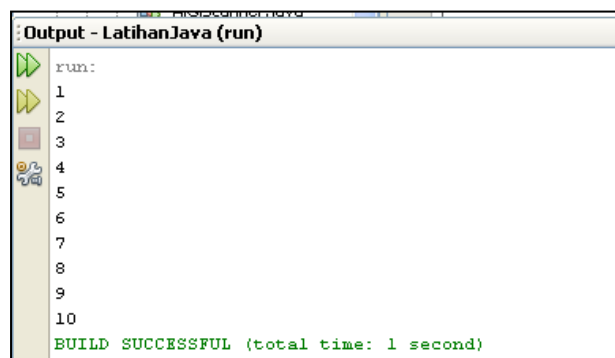
Bentuk umum perulangan do - while, dengan lebih dari perintah / pernyataan, sebagai berikut:

```
do
{
Pernyataan / perintah ;
Pernyataan / perintah ;
}
while ( syarat );
```

Sebagai contoh program menerapkan perintah perulangan do - while, selengkapnya seperti dibawah ini:

```
7  /**
8   *
9   * @author rony
10  */
11  public class ContohDoWhile {
12      public static void main(String[] args)
13      {
14          int bil=1;
15          do
16          {
17              System.out.println(bil);
18              ++bil;
19          }
20          while(bil<=10);
21      }
22  }
```

Output yang dihasilkan dari program ContohDoWhile.java diatas, seperti dibawah ini :



```
run:
1
2
3
4
5
6
7
8
9
10
BUILD SUCCESSFUL (total time: 1 second)
```

Latihan

1. Buatlah program untuk menghitung 10 deret bilangan genap dengan hasilnya :
 $2 + 4 + 6 + 8 + 10 + 12 + 14 + 16 + 18 + 20 = 110$
2. Buatlah program untuk menghitung 10 deret bilangan ganjil dengan hasilnya :
 $1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 + 17 + 19 = 100$
3. Buatlah program untuk menampilkan deret fibonanci, seperti dibawah ini :
1, 1, 2, 3, 5, 8, 13, 21
4. Buatlah program untuk menampilkan bilangan prima, seperti dibawah ini :
2, 3, 5, 7, 11, 13, 17, 19

4.6. Array Pada Pemrograman Java

4.6.1. Array Berdimensi Satu

Variabel Larik atau lebih dikenal dengan ARRAY adalah adalah Tipe terstruktur yang terdiri dari sejumlah komponen-komponen yang mempunyai tipe yang sama. Sebelum digunakan, variabel array perlu dideklarasikan terlebih dahulu. Cara mendeklarasikan variabel array sama seperti deklarasi variabel yang lainnya, hanya saja diikuti oleh suatu indek yang menunjukkan jumlah maksimum data yang disediakan.

Deklarasi Array Bentuk Umum pendeklarasian array :

```
tipe_data[] nama_var_array;
```

```
nama_var_array = new tipe_data[ukuran];
```

Atau bisa juga secara langsung seperti dibawah ini :

```
tipe_data[] nama_var_array = new tipe_data[ukuran];
```

Keterangan :

Type Data :Untuk menyatakan type data yang digunakan.

Ukuran :Untuk menyatakan jumlah maksimum elemen array.

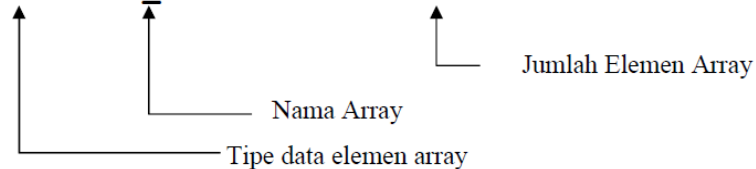
Berikut contoh pendeklarasian array dan membuat objek array;

```
int[] nil_akhir;
```

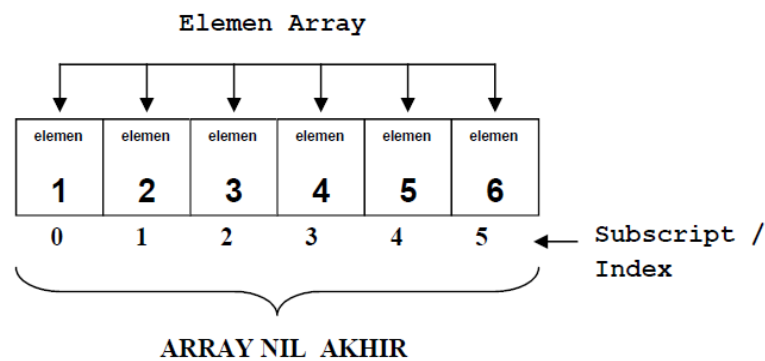
```
nil_akhir = new int[6];
```

Contoh Pendeklarasian Array sekaligus membuat objek array.

```
int[] nil_akhir = new int[6];
```



Pada pendeklarasian diatas, variabel `nil_akhir` sebagai array-of-int, banyak elemen yang dapat ditampung sebanyak 6 (enam) elemen. Suatu array dapat digambarkan sebagai kotak panjang yang berisi kotak-kotak kecil didalam kotak panjang tersebut.



a. Mengakses Array Berdimensi Satu

Suatu array, dapat diakses dengan menggunakan subscript atau indexnya. Bentuk umum pengaksesan dengan bentuk :

```
nama_var_array[nomor_index_array];
```

Contoh :

```
Nil_Akhir[3];
```

```
Nil_Akhir[1];
```

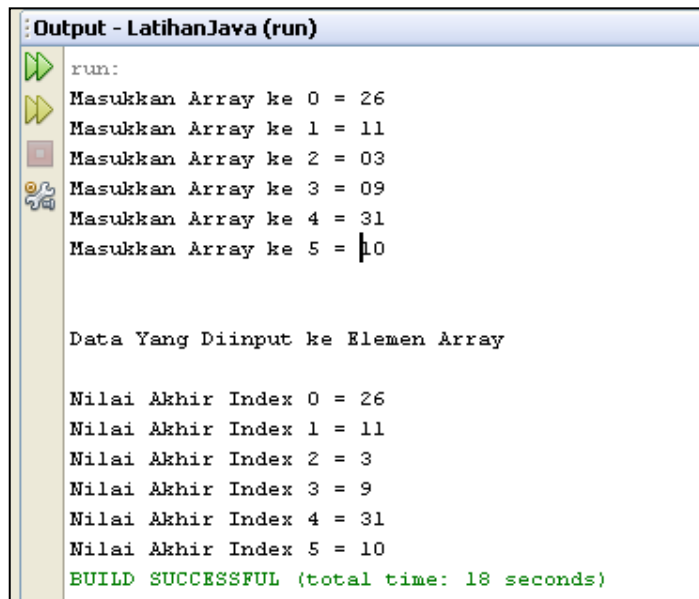
Sebagai contoh program memasukan data kedalam elemen array dan mengambil data dari dalam elemen array, secara selengkapnya seperti dibawah ini:


```

7 import java.util.Scanner;
8
9 /**
10  *
11  * @author rony
12  */
13 public class LatArray1 {
14     public static void main(String[] args)
15     {
16         int i;
17         int[] nil_akhir;
18         // deklarasi variabel array
19         nil_akhir = new int[6]; // membuat objek array
20         Scanner input = new Scanner(System.in);
21         for(i=0; i<6; i++)
22         {
23             System.out.print("Masukkan Array ke " + i + " = ");
24             nil_akhir[i] = input.nextInt();
25         }
26         System.out.println("\n\nData Yang Diinput ke Elemen Array \n");
27         for(i=0; i<6; i++)
28         {
29             System.out.print("Nilai Akhir Index " + i );
30             System.out.println(" = " + nil_akhir[i]);
31         }
32     }
33 }

```

Output yang dihasilkan dari program LatArray1.java diatas, seperti dibawah ini :



```

Output - LatihanJava (run)
run:
Masukkan Array ke 0 = 26
Masukkan Array ke 1 = 11
Masukkan Array ke 2 = 03
Masukkan Array ke 3 = 09
Masukkan Array ke 4 = 31
Masukkan Array ke 5 = 10

Data Yang Diinput ke Elemen Array

Nilai Akhir Index 0 = 26
Nilai Akhir Index 1 = 11
Nilai Akhir Index 2 = 3
Nilai Akhir Index 3 = 9
Nilai Akhir Index 4 = 31
Nilai Akhir Index 5 = 10
BUILD SUCCESSFUL (total time: 18 seconds)

```

b. Inisialisasi Array Berdimensi Satu

Inisialisasi adalah memberikan nilai awal terhadap suatu variabel. Bentuk pendefinisian suatu array dapat dilihat dari contoh berikut :

tipe_data[] nama_array = { nilai array };

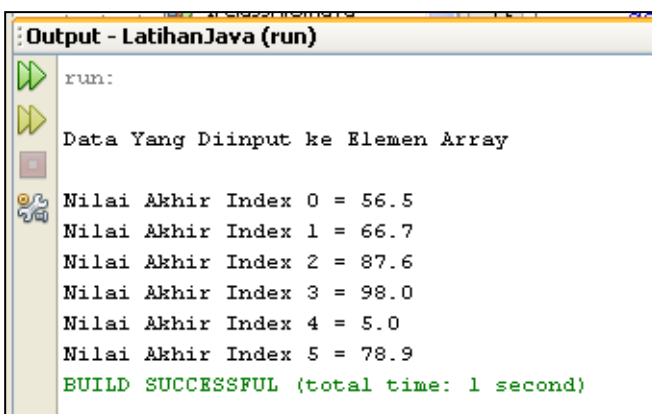
Contoh:

float[] nilai= {56.5, 66.7, 87.45, 98,5, 78.9 };

Sebagai contoh program memasukan data kedalam elemen array dengan cara inisialisasi objek array dan mengambil data dari dalam elemen array, secara selengkapny seperti dibawah ini:

```
6
7  /**
8   *
9   * @author rony
10  */
11  public class InisialisasiArray1 {
12      public static void main(String[] args)
13      {
14          int i;
15          double[] nil_akhir = {56.5, 66.7, 87.6, 98.5, 78.9, 85.4};
16
17          System.out.println("\nData Yang Diinput ke Elemen Array \n");
18          //menampilkan data dari elemen array
19          for(i=0; i<6; i++)
20          {
21              System.out.print("Nilai Akhir Index " + i );
22              System.out.println(" = " + nil_akhir[i]);
23          }
24      }
25  }
```

Output yang dihasilkan dari program Lat602.java diatas, seperti dibawah ini :



```
Output - LatihanJava (run)
run:
Data Yang Diinput ke Elemen Array
Nilai Akhir Index 0 = 56.5
Nilai Akhir Index 1 = 66.7
Nilai Akhir Index 2 = 87.6
Nilai Akhir Index 3 = 98.0
Nilai Akhir Index 4 = 5.0
Nilai Akhir Index 5 = 78.9
BUILD SUCCESSFUL (total time: 1 second)
```

4.6.2. Array dimensi dua

Array dimensi dua tersusun dalam bentuk baris dan kolom, dimana indeks pertama menunjukan baris dan indeks kedua menunjukan kolom. Array dimensi dua dapat digunakan seperti pendataan penjualan, pendataan nilai dan lain sebagainya.

Bentuk Umum pendeklarasian array :

Deklarasi Array Bentuk Umum pendeklarasian array :

```
tipe_data[][] nama_var_array;
nama_var_array = new tipe_data[ukuran_baris][ukuran_kolom];
```

Atau bisa juga secara langsung seperti dibawah ini :

```
tipe_data[][] nama_var_array = new tipe_data[ukuran_baris][ukuran_kolom];
```

Keterangan :

Type Data : Untuk menyatakan type data yang digunakan.

Ukuran_baris : Untuk menyatakan jumlah maksimum elemen baris dalam array.

Ukuran_kolom : Untuk menyatakan jumlah maksimum elemen kolom dalam array.

Berikut contoh pendeklarasian array dan membuat objek array;

```
int[][] nil_akhir;  
nil_akhir = new int[6][3];
```

Contoh Pendeklarasian Array sekaligus membuat objek array.

```
int[][] nil_akhir = new int[6][3];
```

Tipe data elemen array

Nama Array

Jumlah Elemen Baris

Jumlah Elemen Array

a. Mengakses Array Berdimensi Dua

Sebagai contoh pendeklarasian dan pengaksesan data, kita gunakan adalah pengolahan data penjualan, berikut dapat anda lihat pada tabel berikut :

No	Tahun Penjualan		
	2001	2002	2003
1	150	159	230
2	100	125	150
3	210	125	156

Jika anda lihat dari tabel diatas maka dapat dituliskan kedalam array dimensi dua berikut :

```
int[][] data_jual = new int[3][3];
```

Tipe data elemen array

Nama variabel Array

Jumlah Baris

Jumlah Kolom

Sebagai contoh program memasukan data kedalam elemen array berdimensi dua dan mengambil data dari dalam elemen array berdimensi dua, secara selengkapnya seperti dibawah ini:

```

4 import java.util.Scanner;
5 /**
6  *
7  * @author rony
8  */
9 public class LatArray2 {
10     public static void main(String[] args)
11     {
12         int i, j;
13         int[][] data_jual;
14         data_jual = new int[3][3];
15         Scanner input = new Scanner(System.in);
16         for(i=0; i<3; i++)
17         {
18             for(j=0; j<3; j++)
19             {
20                 System.out.print("Masukkan Data Jual ke ["+i+"] ["+j+"] =");
21                 data_jual[i][j] = input.nextInt();
22             }
23             System.out.println();
24         }
25         System.out.println("\n\nData Jual Yang Diinput ke Elemen Array \n");
26         for(i=0; i<3; i++)
27         {
28             for(j=0; j<3; j++)
29             {
30                 System.out.print("Nilai Data Jual ["+i+"] ["+j+"] ");
31                 System.out.println(" = " + data_jual[i][j]);
32             }
33             System.out.println();
34         }
35     }
36 }

```

Output yang dihasilkan dari program LatArray2.java diatas, seperti dibawah ini :

```

Output - LatihanJava (run)
run:
Masukkan Data Jual ke [0][0] =120
Masukkan Data Jual ke [0][1] =90
Masukkan Data Jual ke [0][2] =65

Masukkan Data Jual ke [1][0] =87
Masukkan Data Jual ke [1][1] =90
Masukkan Data Jual ke [1][2] =45

Masukkan Data Jual ke [2][0] =86
Masukkan Data Jual ke [2][1] =73
Masukkan Data Jual ke [2][2] =103

Data Jual Yang Diinput ke Elemen Array

Nilai Data Jual [0][0] = 120
Nilai Data Jual [0][1] = 90
Nilai Data Jual [0][2] = 65

Nilai Data Jual [1][0] = 87
Nilai Data Jual [1][1] = 90
Nilai Data Jual [1][2] = 45

Nilai Data Jual [2][0] = 86
Nilai Data Jual [2][1] = 73
Nilai Data Jual [2][2] = 103

BUILD SUCCESSFUL (total time: 51 seconds)

```

b. Inisialisasi Array Berdimensi Dua

Suatu array, dapat diakses dengan menggunakan subscript atau indexnya. Bentuk umum pengaksesan dengan bentuk :

```
tipe_data[][] nama_array = { nilai array };
```

Contoh:

```
int[][] nil_akhir = {{150, 159, 230}, {100,125,150}, {210,125,156}};
```

Sebagai contoh program memasukan data kedalam elemen array dengan cara inisialisasi objek array dan mengambil data dari dalam elemen array, secara selengkapnya seperti dibawah ini:

```
7  /**
8   *
9   * @author rony
10  */
11  public class InisialisasiArray2 {
12      public static void main(String[] args)
13      {
14          int i, j;
15          int[][] nil_akhir = {{150, 159, 230}, {100,125,150}, {210,125,156}};
16
17          System.out.println("Data Yang Diinput ke Elemen Array \n");
18          //menampilkan data dari elemen array
19          for(i=0; i<3; i++)
20          {
21              for(j=0; j<3; j++)
22              {
23                  System.out.print("Nilai Akhir Index ["+i+"]["+j+"] = ");
24                  System.out.println(" = " + nil_akhir[i][j]);
25              }
26          }
27      }
28  }
```

Output yang dihasilkan dari program InisialisasiArray2.java diatas, seperti dibawah ini :

```
Output - LatihanJava (run)
run:
Data Yang Diinput ke Elemen Array
Nilai Akhir Index [0][0] = = 150
Nilai Akhir Index [0][1] = = 159
Nilai Akhir Index [0][2] = = 230
Nilai Akhir Index [1][0] = = 100
Nilai Akhir Index [1][1] = = 125
Nilai Akhir Index [1][2] = = 150
Nilai Akhir Index [2][0] = = 210
Nilai Akhir Index [2][1] = = 125
Nilai Akhir Index [2][2] = = 156
BUILD SUCCESSFUL (total time: 1 second)
```

BAB V

CONTOH LATIHAN KASUS JAVA

5.1. Kopetensi Dasar

Pada pembahasan Bab 5 ini penulis mengajak mendiskusikan mengenai studi kasus yang yang harus diselesaikan dengan Bahasa Pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa atau pembaca bisa menganalisa dan menyelesaikan dengan bahasa pemrograman Java. Penulis berharap, diakhir pembahasan, para pembaca bisa :

1. Penerapan Program Java dalam Studi Kasus
2. Mahasiswa dapat mempraktekkan Program sebuah kasus IF Majemuk

5.2. Contoh Kasus Array pertama

```
package array;
```

```
import java.io.*;
```

```
class KasusArray
```

```
{
```

```
    public static void main(String[] args) throws IOException
```

```
    {
```

```
        InputStreamReader Keyreader = new InputStreamReader(System.in);
```

```
        BufferedReader input = new BufferedReader(Keyreader);
```

```
        int i, j;
```

```
        double total=0;
```

```
        String [] jenis, ukuran,nama;
```

```
        int [] jumbel;
```

```
        double [] harga, jumhar, pajak, tobar;
```

```
        jenis = new String [10];
```

```
        ukuran = new String [10];
```

```
        jumbel = new int [10];
```

```
        harga = new double [10];
```

```
        jumhar = new double [10];
```

```
        pajak = new double [10];
```

```
        tobar = new double [10];
```

```
        nama=new String[10];
```

```
        System.out.print("Masukkan Jumlah Data : ");j=Integer.parseInt(input.readLine());
```

```
        for (i=1;i<=j;i++)
```

```
        {
```

```
            System.out.println("Data ke - " + i);
```

```
            System.out.print("Merk Helm [KYT/BMC] : ");jenis[i]=input.readLine();
```

```
            System.out.print("Jumlah                Beli                : ");
```

```
            ");jumbel[i]=Integer.parseInt(input.readLine());
```

```
            System.out.print("Ukuran : ");ukuran[i]=input.readLine();
```

```
            System.out.println("");
```

```
            if (jenis[i].equalsIgnoreCase("1"))
```

```
            {
```

```
                nama[i]="kyt";
```

```
                if (ukuran[i].equalsIgnoreCase("S"))
```

```

{
    harga[i] = 80000;
}
else if (ukuran[i].equalsIgnoreCase("M"))
{
    harga[i] = 70000;
}
else
{
    harga[i] = 60000;
}
}
else
{
    if (ukuran[i].equalsIgnoreCase("S"))
    {
        harga[i] = 90000;
    }
    else if (ukuran[i].equalsIgnoreCase("M"))
    {
        harga[i] = 85000;
    }
    else
    {
        harga[i] = 75000;
    }
}

jumhar[i]=harga[i]*jumbel[i];
pajak[i]=jumhar[i]*0.1;
tobar[i]=jumhar[i]+pajak[i];
total=total+tobar[i];
}

System.out.println("TOKO HELM XXX");
System.out.println("");

for (i=1;i<=j;i++)
{
    System.out.println("Data ke - " + i);
    System.out.println("Jenis Helm : " + jenis[i]);
    System.out.println("Ukuran : " + ukuran[i]);
    System.out.println("Harga : " + harga[i]);
    System.out.println("Jumlah Harga : " + jumhar[i]);
    System.out.println("Pajak : " + pajak[i]);
    System.out.println("Total Bayar : " + tobar[i]);
    System.out.println("");
}

System.out.println("Harga Semuanya adalah " + total);
}
}

```

5.3. Contoh kasus array kedua

```
import java.util.*;
class ArrayPlg
{
    public static void main(String[] args)
    {
        Scanner input=new Scanner(System.in);

        int i,jd;

String [] nm=new String[10];
        String [] kp=new String[10];
        String [] almt=new String[10];
        String tanya="";

        do{

            System.out.print("masukan jumlah data :");
            jd=input.nextInt();
            for(i=1;i<=jd;i++)
            {
                System.out.println(" Data Ke- " +i);
                System.out.print(" kode pelanggan :");
                kp[i]=input.next();
                System.out.print(" nama pelanggan :");
                nm[i]=input.next();
                System.out.print(" alamat pelanggan :");
                almt[i]=input.next();

            }

        }
```



```

System.out.println("\n\n      DATA PELANGGAN PT. SENTOSA JAYA ");
System.out.println("      JL. Sagu No.27A Kebagusan Jakarta selatan ");
System.out.println("=====");
//System.out.println("No \t Kode Pelanggan \t Nama Pelanggan\t\t Alamat");
//System.out.println("=====");
for (i=1;i<=jd;i++)
{

System.out.println(" Data Ke- " +i);
System.out.println("Kode pelanggan : " +kp[i]);
System.out.println("nama pelanggan : " +nm[i]);
System.out.println("Alamat pelanggan : " +almt[i]);
System.out.println("=====");
//System.out.println(i+"\t\t"+kp[i]+" \t\t\t\t"+nm[i]+" \t\t\t\t"+almt[i]);

}
//System.out.println("=====");
System.out.print("Mau input data lagi :");tanya=input.next();
}while(tanya.equals("Y") || tanya.equals("y"));
}
}

```

LATIHAN

LATIHAN SOAL ARRAY

Buatlah Program Penjualan barang pada PT. Permata Pratama dengan Layout Sebagai berikut :

Tampilan Input :

```

                PT. PERMATA "PRATAMA"
+++++
Masukan Nama Petugas      :
Tanggal                    :

Jumlah Data yang akan di masukan      :
-----
                Data Ke-
                Kode Barang      :
                Jumlah           :
-----

```

Tampilan Output:**PT. PERMATA “PRATAMA”**

+++++

Nama Petugas : <output> Tanggal : <output>

Jumlah Data yang di masukan : <output>

Data Ke	Kode Barang	Nama Barang	Harga Barang	Jumlah Barang	Total Harga
---------	-------------	-------------	--------------	---------------	-------------

1	xxx	xxxxxx	Rp. xxxx	xxxxx	Rp. xxxx
2	xxx	xxxxxx	Rp. xxxx	xxxxx	Rp. xxxx
3	xxx	xxxxxx	Rp. xxxx	xxxxx	Rp. xxxx

Total Pendapatan Pada tanggal <output> adalah sebesar Rp. <output>

Ketentuan soal :

- Gunakan Array untuk memasukan data sesuai dengan jumlah data yang di masukan
- Untuk nama barang dan harga barang dihasilkan dari masukan kode barang, dengan ketentuan :

Kode Barang	Nama Barang	Harga Barang
P001	Printer	700000
V001	VGA Card	75000
M001	Motherboard	950000

- Total didapatkan dari harga barang dikalikan dengan jumlah barang
- Total pendapatan merupakan hasil dari penjumlahan total harga data yang di masukan

BAB VI

MEMBUAT FORM SECARA KONVENSIONAL

6.1. Kopetensi Dasar

Pada pembahasan Bab 6 ini menjelaskan pembuatan form aplikasi form sederhana dengan full script bahasa Pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa dapat:

1. Mengetahui dan memahami pembuatan form Java di Netbeans
2. Mempraktekan pembuatan form dengan full script
3. Mengetahui dan memahami pembuatan form Java di Netbeans
4. Mempraktekan pembuatan form dengan full script

6.2. Gui(Graphical User Interface)

GUI (Graphical User Interface), memungkinkan user untuk berinteraksi dengan komputer secara lebih baik. Pengguna dapat berinteraksi dengan berbagai elemen grafis seperti button, dialog box, menu dan sebagainya. Sejak tahun 1994 Sun MicroSystem berupaya untuk membuat JAVA memiliki perangkat berhaya desktop dengan ToolKitJAVA yang dikenal sebagai AWT (Abstract Windowing Toolkit). Tahun 1997, muncul Java Foundation Class (JFC) untuk kepentingan pembuatan aplikasi Window yang merupakan pengembangan dari AWT. JFC mempunyai kode swing yang terdapat pada package yang disediakan JAVA SDK versi 1.2 ke atas (JAVA 2). JAVA menyediakan dua graphical library yaitu AWT dan Swing.

Contoh: `import java.awt.*;`
 `import java.awt.event.*;`
 `import javax.swing.*;`

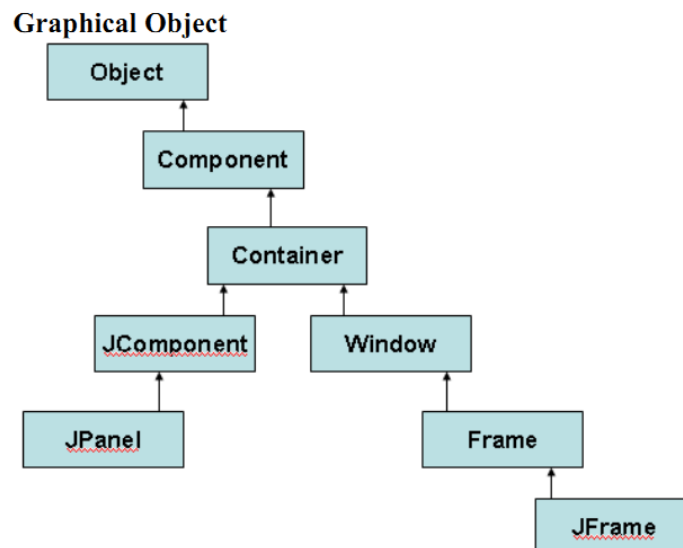
ada beberapa jenis graphical object, antara lain:

1. Container (tempat), obyek yang dapat menampung komponen grafis dan tempat yang lebih kecil. Contoh: frames, panels.
2. Component, obyek grafis yang berfungsi untuk menampilkan data, menerima masukan, atau menunjukkan suatu kondisi. Contoh: buttons, labels, textfields.
3. Event, obyek yang merepresentasikan suatu kejadian. Contoh: mouseMove, keyPressed.
4. Listener, suatu obyek yang menunggu suatu kejadian khusus dan akan beraksi (melakukan sesuatu) jika kejadian tersebut terjadi.

Untuk pembuatan form menggunakan bahasa pemrograman java ada dua cara yaitu cara konvensional (full script) dan menggunakan palette. Pembuatan form menggunakan cara konvensional (full script) merupakan cara yang digunakan programmer untuk membuat

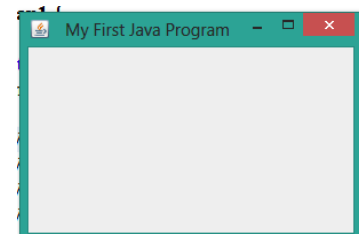
aplikasi dengan menulis source code untuk pembuatan objek swing yang akan digunakan, seperti membuat label, textfield, dll.

Untuk pembuatan form menggunakan pallets, adalah cara pembuatan form dengan merancang masing-masing objek pallets yang dibutuhkan kedalam form yang sudah disediakan oleh Netbeans.



Contoh pembuatan form dengan menggunakan cara konvensional (full script). Langkah-langkah pembuatannya antara lain:

1. Buatlah project terlebih dahulu
2. Buat package untuk latihan anda
3. Klik kanan di package anda dan pilih New Java Class.
4. Beri nama latihan1.java → Ok
5. Ketik source code berikut:



```
10  */
11  import javax.swing.*;
12
13  class latihan1 {
14
15      public static void main(String[] args) { //agar dapat di run sebagai class utama
16          JFrame myWindow;
17
18          myWindow = new JFrame(); //pembuatan object
19          myWindow.setSize(300, 200); //setting ukuran form
20          myWindow.setTitle("My First Java Program"); //judul untuk form
21          myWindow.setVisible(true); // untuk menampilkan form
22      }
23  }
```

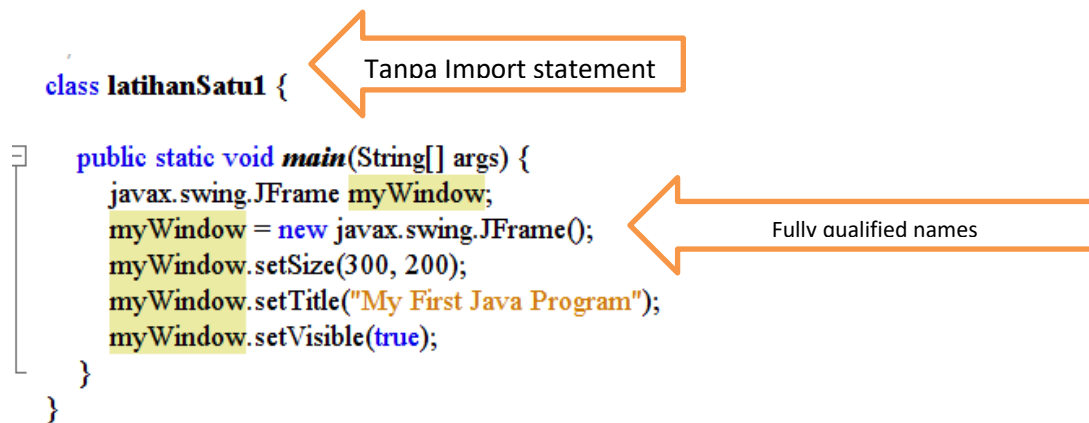


Sebuah package dapat termasuk subpackage, sehingga membentuk hirarki package. Pada saat menunjuk ke sebuah class yang ada di dalam nested package, kita gunakan multiple dots. Sebagai contoh, kita menulis

javax.swing.JFrame;

untuk merujuk ke class JFrame di package javax.swing; itulah, swing package yang ada di dalam java package. Notasi Dot (“.”) dengan semua nama package yang dimiliki class biasanya disebut the class’s fully qualified name. Dengan menggunakan class tersebut acapkali sangat tidak praktis. Kita dapat menggunakan import statement untuk menghindari masalah ini. Berikut ini source code tanpa import statement:

```
class latihanSatu1 {  
    public static void main(String[] args) {  
        javax.swing.JFrame myWindow;  
        myWindow = new javax.swing.JFrame();  
        myWindow.setSize(300, 200);  
        myWindow.setTitle("My First Java Program");  
        myWindow.setVisible(true);  
    }  
}
```



Berikutnya kita akan membuat sebuah form sederhana dengan menambahkan komponen label atau dikenal dengan JLabel dan juga komponen textfield (JTextField)

```

import javax.swing.*;
import java.awt.*;

public class DemoGui2 extends JFrame{
    JPanel jp=new JPanel();
    JTextField jt=new JTextField();
    JLabel jl=new JLabel("contoh label");

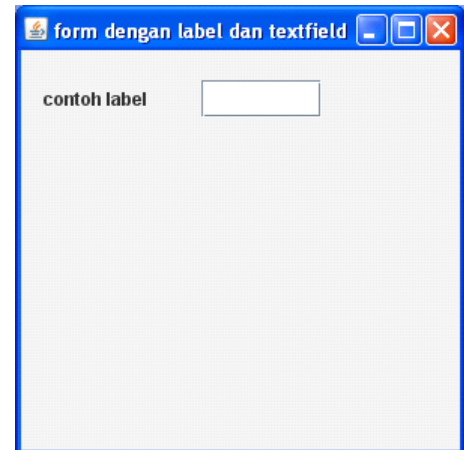
    public DemoGui2(){
        add(jl);
        add(jt);
        add(jp);
        jl.setBounds(15, 20, 80, 25);
        jt.setBounds(120, 20, 80, 25);
    }

    public static void main(String[] args){
        DemoGui2 demo2=new DemoGui2();

        demo2.setTitle("form dengan label dan textfield");
        demo2.setSize(300,300);
        demo2.setVisible(true);

    }
}

```



Untuk latihan berikutnya kita akan selain textfield dan label kita akan menambahkan komponen Button (JButton), berikut contohnya

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class ContohGUI extends JFrame{

    JPanel pl=new JPanel();
    JLabel lbl1=new JLabel("String I");
    JLabel lbl2=new JLabel("String II");
    JLabel lbl3=new JLabel("Hasil");

    JTextField t1=new JTextField();
    JTextField t2=new JTextField();
    JTextField t3=new JTextField();

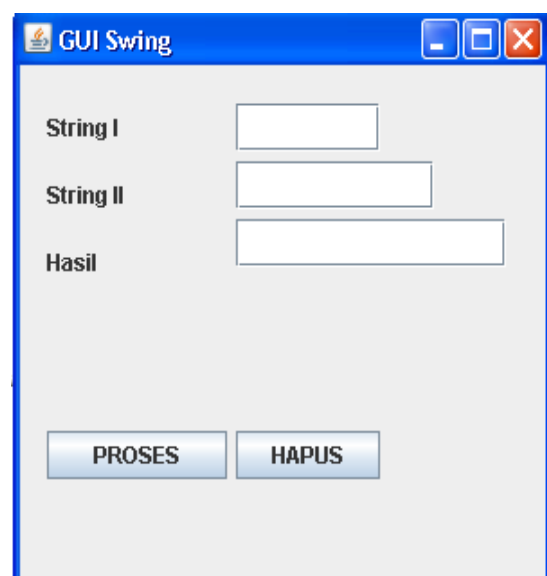
    JButton b1=new JButton("PROSES");
    JButton b2=new JButton("HAPUS");

    public ContohGUI(){
        add(lbl1);
        add(lbl2);
        add(lbl3);

        add(t1);
        add(t2);
        add(t3);

        add(b1);
    }
}

```



```

        add(b2);
        add(pl);

setTitle("GUI Swing");
setBounds(100,100,300,300);
lbl1.setBounds(15,20,80,25); //15 jarak antr pinggir
lbl2.setBounds(15,55,125,25);
lbl3.setBounds(15,90,125,25);

t1.setBounds(120,20,80,25);
t2.setBounds(120,50,110,25);
t3.setBounds(120,80,150,25);

b1.setBounds(15,190,100,25);
b2.setBounds(120,190,80,25);

setDefaultCloseOperation(EXIT_ON_CLOSE);

b1.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        kirimdata();}
});
b2.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent e){
        hapusdata();}
});
}

public void hapusdata(){
    t1.setText("");
    t2.setText("");
    t3.setText("");
}

}
public static void main(String[] args) {
    ContohGUI demo = new ContohGUI();
    demo.setVisible(true);
}
}

```

Pengembangan berikutnya kita akan mencoba untuk membuat aplikasi form sederhana dengan object swing didalamnya. Berikut contoh source code untuk penambahan kata ke dalam sebuah TextArea:

```

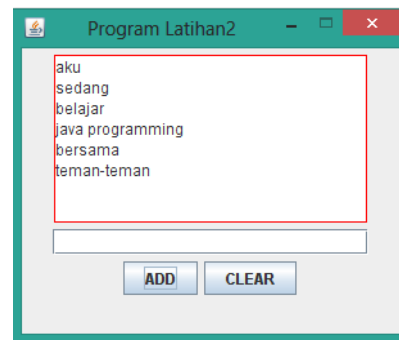
Latihan2.java
Source History
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class Latihan2 extends JFrame implements ActionListener {

    private static final int FRAME_WIDTH = 300;
    private static final int FRAME_HEIGHT = 250;
    private static final int FRAME_X_ORIGIN = 150;
    private static final int FRAME_Y_ORIGIN = 250;
    private static final String EMPTY_STRING = "";
    private static final String NEWLINE = System.getProperty("line.separator");
    private JButton clearButton;
    private JButton addButton;
    private JTextField inputLine;
    private JTextArea textArea;

    public static void main(String[] args) {
        Latihan2 frame = new Latihan2();
        frame.setVisible(true);
    }

```



```

public Latihan2() {
    Container contentPane;
    //setting properties frame
    setSize(FRAME_WIDTH, FRAME_HEIGHT);
    setResizable(false);
    setTitle("Program Latihan2");
    setLocation(FRAME_X_ORIGIN, FRAME_Y_ORIGIN);
    contentPane = getContentPane();
    contentPane.setLayout(new FlowLayout());
    textArea = new JTextArea();
    textArea.setColumns(22);
    textArea.setRows(8);
    textArea.setBorder(BorderFactory.createLineBorder(Color.RED));
    textArea.setEditable(false);
    contentPane.add(textArea);
    inputLine = new JTextField();
    inputLine.setColumns(22);
    contentPane.add(inputLine);
    inputLine.addActionListener(this);

    //membuat dua button ke dalam frame
    addButton = new JButton("ADD");
    contentPane.add(addButton);

    clearButton = new JButton("CLEAR");
    contentPane.add(clearButton);

    //membuat action listener untuk kedua button
    clearButton.addActionListener(this);
    addButton.addActionListener(this);

    //tutup form saat [x] diklik
    setDefaultCloseOperation(EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent event) {
    if (event.getSource() instanceof JButton) {
        JButton clickedButton = (JButton) event.getSource();
        if (clickedButton == addButton) {
            addText(inputLine.getText());
        } else {
            clearText();
        }
    } else { //event untuk input
        addText(inputLine.getText());
    }
}

```

```

private void addText(String newline) {
    textArea.append(newline + NEWLINE);
    inputLine.setText("");
}

private void clearText() {
    textArea.setText(EMPTY_STRING);
    inputLine.setText(EMPTY_STRING);
}

```

Ketika kita menjalankan class Latihan2 dan menambahkan beberapa teks ke dalam TextArea apa yang terjadi? Tinggi TextArea akan menyesuaikan jumlah teks yang sudah diinput ke dalamnya. Demikian juga, TextArea akan melebar ketika kita memasukan sebaris text dengan

panjang yang melebihi lebar TextArea. Namun itu bukanlah hal yang diinginkan dalam class ini. Maka cara mudah untuk mengatasinya dengan instance dari `javax.swing.JScrollPane` yang akan menambahkan vertical dan horizontal scroll bars ketika diperlukan.

BAB VII

REVIEW MATERI

- Review Teori dan Presentasi Studi Kasus yang diberikan Instruktur sebelumnya.
- Mahasiswa dapat Review Teori dan Presentasi Studi Kasus yang diberikan Instruktur.

BAB VIII

UJIAN TENGAH SEMESTER

BAB IX

MEMBUAT FORM DENGAN JFRAME FORM DAN OBJECT SWING

9.1. Kopetensi Dasar

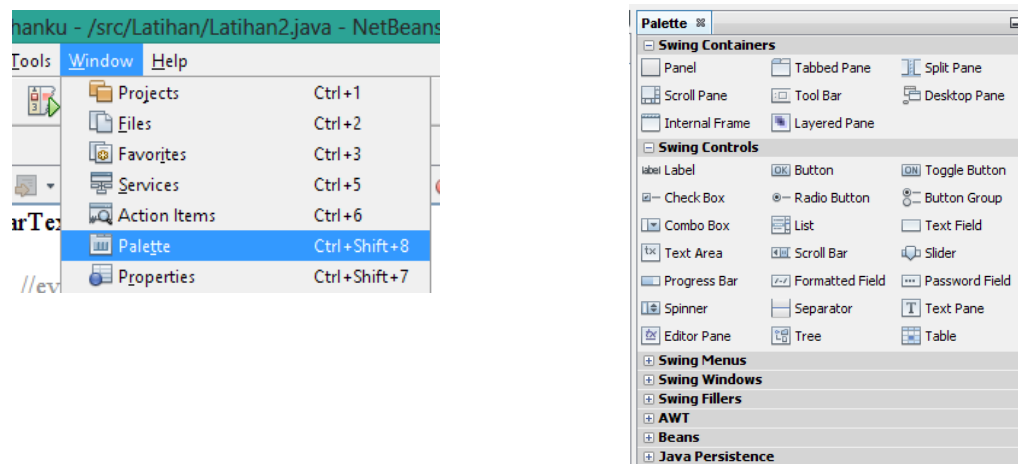
Pada pembahasan Bab 9 ini Membuat form dan implementasi OOP, Membuat form aplikasi sederhana JFrame Form dengan object swing pada bahasa Pemrograman Java. Kompetensi dasar secara umum, agar mahasiswa dapat:

1. Membuat form aplikasi dengan menerapkan instansiasi objek
2. Menerapkan insert code untuk getter dan setter
3. Membuat form dan implementasi logika
4. Membuat form aplikasi dengan mengimplementasikan logika if (majemuk dan bersarang)

9.2. Pembuatan Form JFrame dan Object SWING

Dalam bab ini akan membahas tentang cara pembuatan form, tetapi pembuatan form di bab ini berbeda dengan bab sebelumnya. Bab sebelumnya sudah menerangkan cara pembuatan form dimana dalam proses pembuatannya menggunakan source code untuk masing-masing object swing pada form.

Netbeans memiliki taskpane palette dimana didalamnya terdapat objek-objek yang kita butuhkan untuk pembuatan form Java. Untuk menampilkan palette di Netbeans dapat memilih menu window → Palette atau dapat menekan Ctrl+Shift+8.

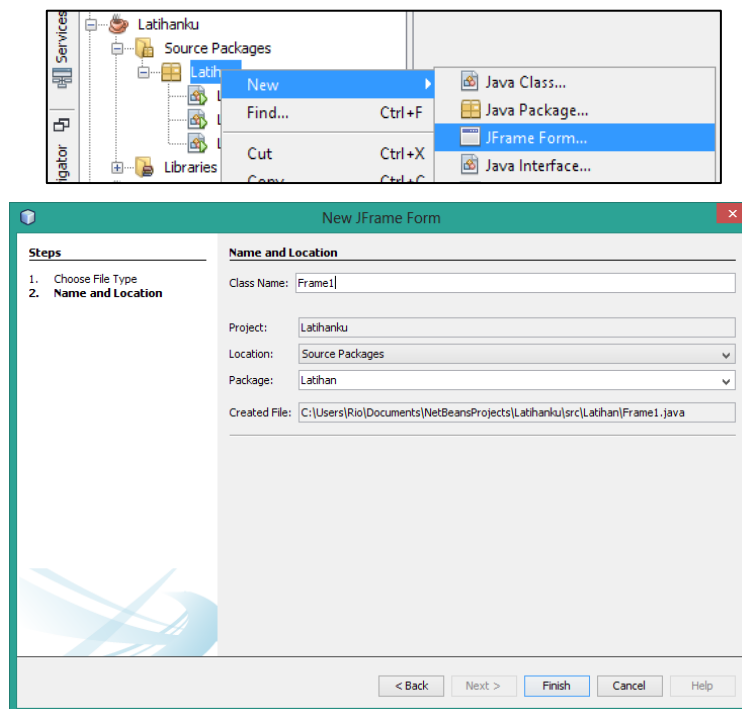


Di dalam palette memiliki beberapa package object yang dapat digunakan untuk pembuatan form. Salah satunya yang sering digunakan adalah package Swing Controls, dan masing-masing package memiliki berbagai macam fungsi yang berbeda-beda tergantung dari aplikasi apa yang akan dibuat.

Untuk pemakaiannya sendiri anda dapat memilih salah satu objek yang akan digunakan dan meletakkannya ke dalam form. Semua object pada palette memiliki properties yang berbeda-beda begitu pula dengan fungsinya.

Pembuatan form dengan menggunakan palette terkesan begitu mudah, karena cukup memilih object lalu meletakkannya ke dalam form. Untuk lebih jelasnya berikut contoh form aplikasi menggunakan JFrame Form dan Palette:

1. Klik kanan pada package yang anda punya lalu pilih New → JFrame Form

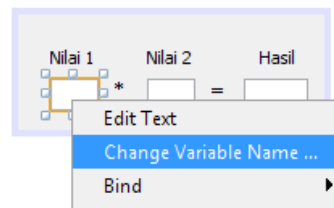


2. Beri nama Frame1. Perlu diketahui pada saat kita membuat JFrame Form, form ini dijadikan juga sebagai main class. Oleh karena itu, nama form yang kita buat adalah sama dengan nama class. Jika form yang kita buat diberi nama Frame1, maka secara otomatis memiliki nama class Frame1.
3. Kemudian rancang form seperti gambar dibawah ini.

Properties object pada form:

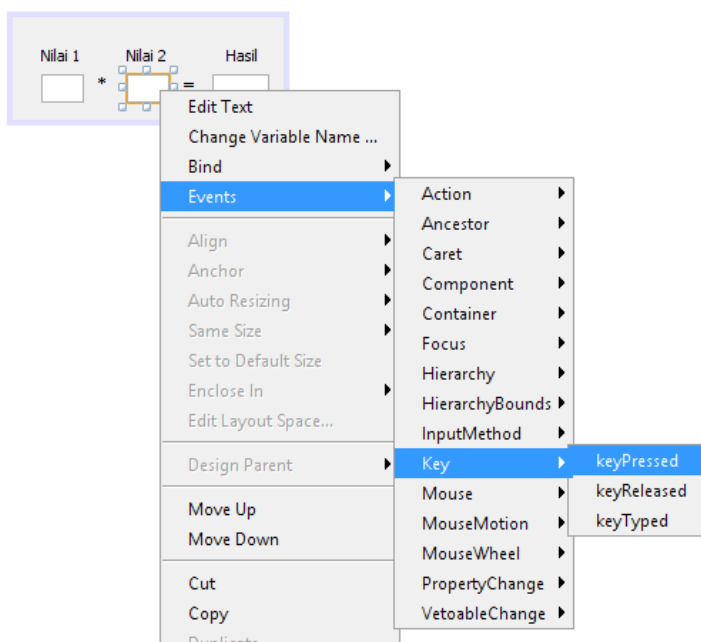
Object	Edit Text	Variable Name
JLabel1	labelNilai1	-
JLabel1	labelNilai2	-
TextField1	-	tNilai1
TextField2	-	tNilai2
TextField3	-	tHasil

Untuk mengubah Text dan/atau variable name masing-masing object, anda dapat mengklik kanan pada object.



Tujuan dari pengubahan text dan/atau variable name ini adalah agar programmer lebih mudah mengingat masing-masing object yang ada di dalam form. Hal ini berpengaruh terhadap source code yang akan diketik nantinya.

Proses dari program ini adalah kita input nilai1 dan nilai2 lalu tekan Enter untuk menampilkan hasilnya. Ada beberapa event yang dapat kita fungsikan di setiap object yang kita buat, dan masing-masing event memiliki fungsinya. Untuk mendapatkan hasil dari perkalian nilai1 dan nilai2, kita membutuhkan event KeyPressed karena disini kita menggunakan tombol Enter.



TextField Nilai2 kita berikan event KeyPressed karena tombol Enter di tekan setelah meng-input Nilai2. Dengan kata lain, proses di program sederhana ini berada di TextField Nilai2 yang kita ubah variable name-nya menjadi tNilai2.

Selanjutnya kita coba untuk pengetikan source code-nya.

Source code Frame1:

```
private void tNilai2KeyPressed(java.awt.event.KeyEvent evt) {  
    // TODO add your handling code here:  
    if (evt.getKeyCode() == KeyEvent.VK_ENTER) {  
        int a = Integer.parseInt(tNilai1.getText());  
        int b = Integer.parseInt(tNilai2.getText());  
        int c = a * b;  
        tHasil.setText(Integer.toString(c));  
    }  
}
```

Konversi nilai agar dapat dikalkulasi

Setiap nilai yang akan dikalkulasikan harus di konversi ke tipe data yang sesuai. Mengapa seperti itu? Karena setiap nilai yang diinput ke dalam TextField secara otomatis memiliki tipe data String, dimana tipe data tersebut tidak dapat dikalkulasikan. Oleh karena itu, sangat diperlukan adanya proses konversi dalam perhitungan ini. Begitu pula dengan pemberian nilai ke TextField dimana nilai tersebut harus berupa String, karena TextField hanya dapat menampung nilai inputan berupa tipe data String saja. Dapat dilihat di baris akhir source code di atas, tHasil akan menerima nilai hasil perkalian nilai1 dan nilai2.

Ada dua jenis peng-konversi-an dalam java. `Integer.parseInt(tNilai1.getText())`, artinya nilai tNilai1 diambil dan dikonversi ke tipe data Integer, hal ini dapat juga disebut sebagai persiapan kalkulasi. Sedangkan `tHasil.setText(Integer.toString(c))`, memiliki makna bahwa TextField tHasil akan diberikan nilai yang diambil dari variable c (variable penampung) yang memiliki tipe data Integer dan dengan method **Integer.toString** nilai tersebut diubah mejadi String agar dapat diterima oleh TextField tHasil.

Catt:

“int a” merupakan pendeklarasian variable secara langsung di dalam event. Pengetikkan tipe data dalam membuat program java memiliki beberapa perbedaan. Sebagai contoh, untuk deklarasi variable tipe data integer menjadi “int a”, namun pada saat proses konversi tipe data, integer diketik “Integer” dengan huruf kapital “I”. Terkecuali untuk tipe data String, pengetikannya harus “String” bukan “string”, baik itu dalam pendeklarasian variable maupun konversi tipe data.

9.3 Membuat Form Dan Implementasi OOP

Java merupakan salah satu bahasa pemrograman berbasis object dan memiliki beberapa karakteristik seperti yang sudah dibahas di bab sebelumnya. Pada saat anda membuat form, dengan sendirinya main class utama akan terbentuk. Dengan begitu kita hanya perlu membuat sub class-nya untuk menerapkan bahasa pemrograman yang berbasis object. Biasanya untuk program java yang kompleks, programmer membutuhkan beberapa sub class. Sub class tersebut dapat membantu programmer dalam mengelompokkan proses yang akan terjadi di main class.

Satu main class bisa saja membutuhkan dua atau lebih sub class dalam pembuatan dan prosesnya. Berikut adalah contoh dimana main class dan subclass dibuat menjadi satu kesatuan program. Langkah-langkah:

1. Membuat JFrame Form (main class) dan beri nama NilaiRata.java. Program sederhana ini akan menghitung nilai rata-rata dari nilai yang diinput.



Jpanel → di panel Swing Container

Properties :

Border→title

border→title :Pilih tombol

Properties object pada form:

Object	Edit Text	Variable Name
JLabel1	Nilai 1	-
JLabel2	Nilai 2	-
JLabel3	Rata-rata	-
JTextField1	-	tNilai1
JTextField2	-	tNilai2
JTextField3	-	tRata
JButton1	Hitung	bHitung
JButton2	Bersih	bBersih
JButton3	Tutup	bTutup

2. Membuat subclass ratarata.java. Di dalam sub class ini terdapat proses yang akan mengoperasikan jalannya program pada Form (main class). Oleh karena itu, kita harus dapat menganalisa variable apa saja yang akan digunakan dalam pembuatan program sederhana ini.

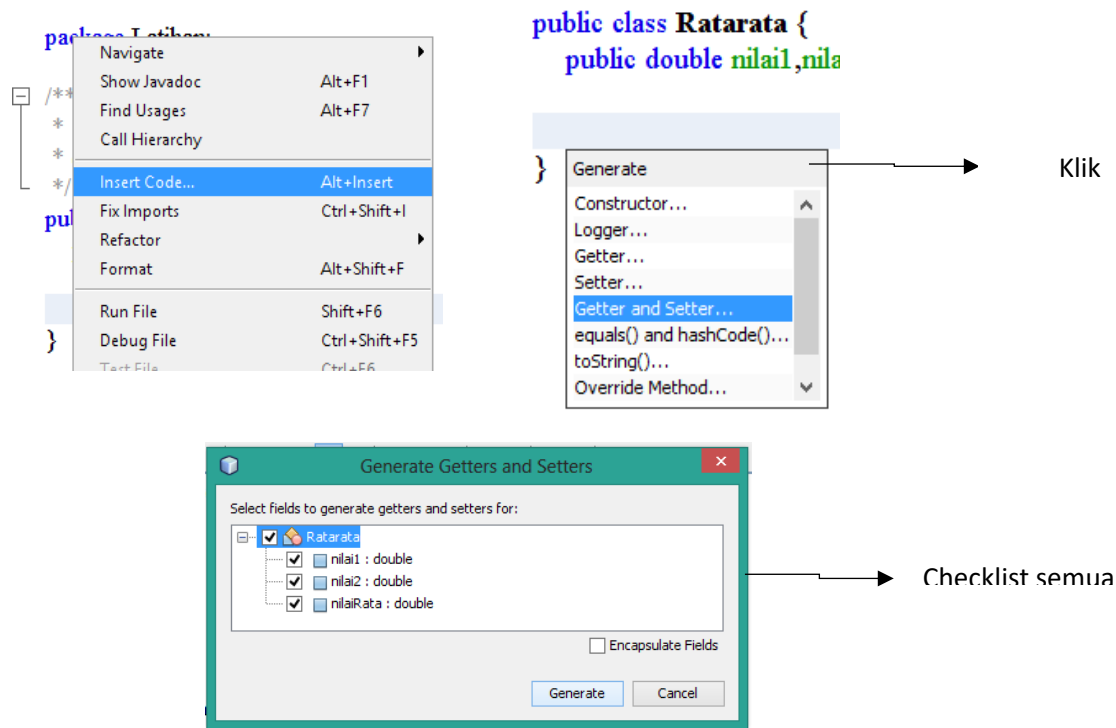

```

package Latihan;

/**
 *
 * @author Rio
 */
public class Ratarata {
    public double nilai1, nilai2, nilaiRata;
}

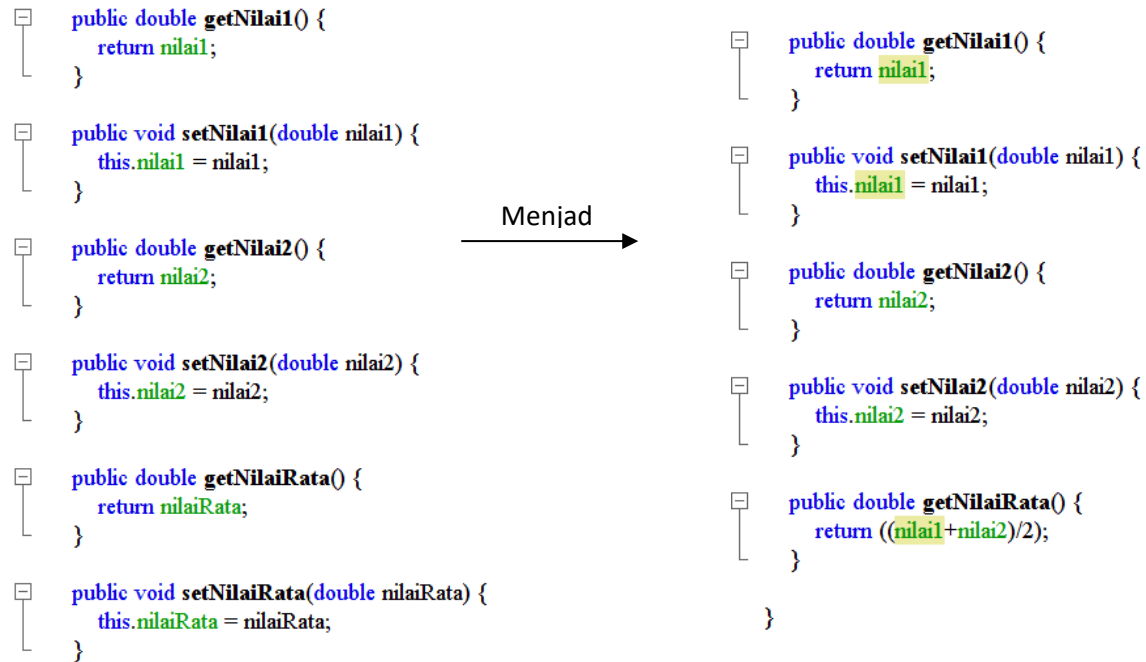
```

Setelah variable ditentukan, maka source code setter dan getter harus dibuat. Agar lebih mudah dalam pembuatan setter dan getter, kita gunakan menu insert code dengan meng-klik kanan pada area source code yang kosong lalu pilih insert code atau dengan shortcut Alt+Insert.

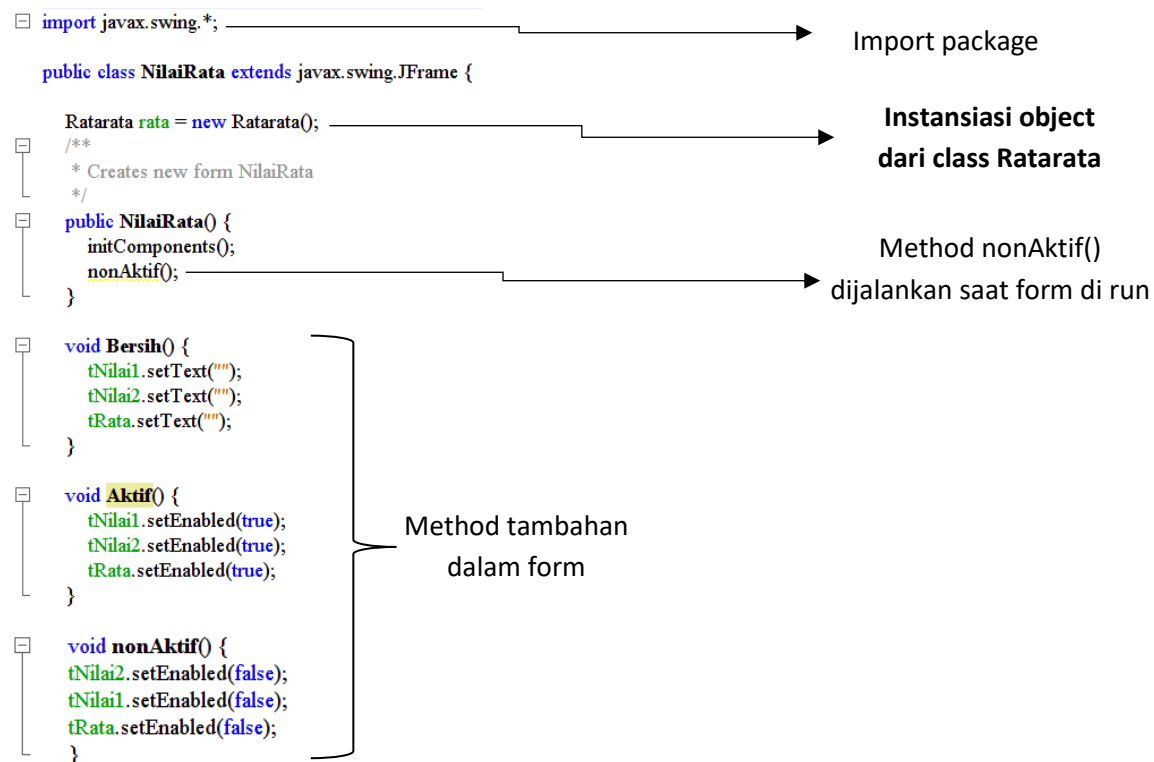
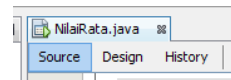


Checklist untuk semua atribut lalu klik generate, maka method masing-masing atribut akan terbentuk secara otomatis. Selanjutnya, tugas anda adalah menambahkan sedikit source code ke masing-masing method yang sudah dibuat dan memodifikasinya agar dapat diproses oleh main class sesuai dengan program yang kita inginkan.

Source code subclass **Ratarata.java**:



Untuk menambahkan atau melihat source code pada form, klik source
Source code form (main class) **NilaiRata.java**:



```

private void bHitungActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    rata.setNilai1(Double.parseDouble(tNilai1.getText()));
    rata.setNilai2(Double.parseDouble(tNilai2.getText()));
    tRata.setText(Double.toString(rata.getNilaiRata()));
}

private void bBersihActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    Bersih();
    Aktif();
    tNilai1.requestFocus();
}

private void bTutupActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    int tutup;
    tutup=JOptionPane.showConfirmDialog(null,"Yakin tutup program?","Konfirmasi",JOptionPane.YES_NO_OPTION);
    if (tutup==JOptionPane.YES_OPTION)
    {dispose();}
    else
    { return;}
}

```

Pemanggilan method

Dilihat dari source code di atas, tombol yang kita ubah variable name-nya menjadi bHitung, bBersih, dan bTutup memiliki source code dengan fungsinya masing-masing. Agar kita dapat menambahkan source code ke dalam tombol, cukup double-click jButton yang ingin kita berikan source code. Atau dapat juga meng-klik kanan jButton → Events → Action → actionPerformed.

```

rata.setNilai1(Double.parseDouble(tNilai1.getText()));

```

Pada code diatas menunjukkan bahwa nilai TextField tNilai1 diambil dan dikonversi ke tipe data double dan diberikan ke method setNilai() yang ada di subclass Ratarata.java untuk ditampung nilainya. Dengan kata lain, sebelum kita memberikan nilai ke method yang ada di subclass, kita harus mengkonversi nilainya sesuai dengan tipe data yang digunakan oleh method tersebut. Jika method yang akan kita gunakan memiliki tipe data double, maka kita harus mengkonversi nilai yang diambil menjadi double.

```

tRata.setText(Double.toString(rata.getNilaiRata()));

```

Dapat dilihat dari code diatas TextField tRata diberikan nilai yang didapat dari method getNilaiRata() dan dikoneversi nilainya dari double menjadi String agar dapat diterima oleh TextField.

BAB X

MEMBUAT FORM DAN IMPLEMENTASI LOGIKA

10.1. Kompetensi Dasar

Pada pembahasan Bab 10 ini Membuat form dan implementasi logika dengan program form dan program class pada bahasa Pemrograman Java. Kompetensi dasar secara umum, agar mahasiswa dapat:

1. Mahasiswa dapat mengerti penerapan fungsi logika (if atau switch case)
2. Mahasiswa dapat membuat aplikasi form java dengan menggunakan sub class dan main class

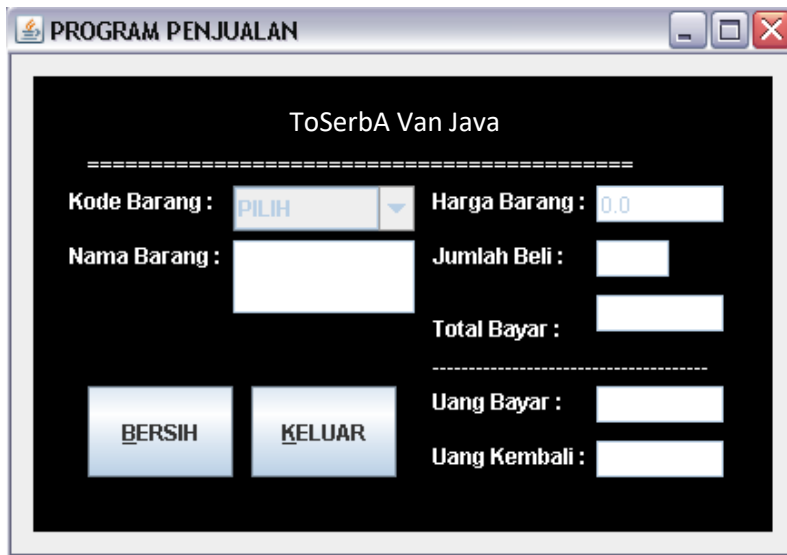
10.2. Pembuatan Form dan Penerapan Logika

Didalam pembuatan program, pasti terdapat logika agar jalannya program lebih baik sesuai dengan keinginan user. Untuk membuat program, anda sebagai programmer tidak bisa berpikir juga sebagai programmer, tetapi anda harus berpikir secara nyata dan berpikir sebagai user. Dengan demikian, anda dapat memahami kemauan user didalam programnya.

Di bab ini akan membahas penerapan logika ke dalam program yang dibuat. Penerapan logika dapat anda buat dengan If atau switch case tergantung dari seberapa kompleks programnya. Untuk bentuk if dan switch case sendiri sudah dibahas pada bab sebelumnya. Oleh karena itu, bab ini langsung membahas tentang cara penerapan logika, instansiasi dan form.

Berikut adalah contoh penerapannya:

1. Buat form dengan nama FormBarang.java dan rancanglah form seperti gambar berikut.



Pada form diatas kita tambahkan object swing baru yaitu JComboBox. JComboBox sering digunakan programmer untuk memilih salah satu item dari beberapa item yang ada. Contoh diatas digunakan untuk memilih kode barang yang akan dibeli.

Properties object di Form:

Object	Edit Text	Variable name
jLabel1	Kode Barang:	-
jLabel2	Nama Barang:	-
jLabel3	Harga Barang:	-
jLabel4	Jumlah Beli:	-
jLabel5	Total Bayar:	-
jLabel6	Uang Bayar:	-
jLabel7	Uang Kembali:	-
jComboBox	-	cbKode
(jTextField1)	-	nb
(jTextField2)	-	thrg
(jTextField3)	-	tjml
(jTextField4)	-	ttot
(jTextField5)	-	tubyr
(jTextField6)	-	tukemb
jButton1	BERSIH	bnew
jButton2	KELUAR	bclose

Setelah pembuatan from (main class) selesai, seperti biasa kita butuh subclass untuk membuat source code proses. Buat class dengan nama Barang.java dan perhatikan source code untuk class Barang.java di bawah ini.

```
public class Barang {  
    public String kd,nm;  
    public int jml;  
    public double hrg,ttl,ub,uk;  
  
    public void setKd(String val){  
        kd=val;  
    }  
    public String getKd(){  
        return kd;  
    }  
}
```

```
public void setSeleksiKd(){  
    if(kd.equalsIgnoreCase("B001")){  
        nm="Pensil";  
        hrg=3000;  
    }else if(kd.equalsIgnoreCase("B002")){  
        nm="Buku";  
        hrg=5000;  
    }else if(kd.equalsIgnoreCase("B003")){  
        nm="penggaris";  
        hrg=1000;  
    }else if(kd.equalsIgnoreCase("B004")){  
        {  
            nm="penghapus";  
            hrg=500;  
        }  
    }else{  
        nm="";  
        hrg=0;  
    }  
}
```

```

void bersih(){
    nb.setText("");
    thrG.setText("");
    tjml.setText("");
    ttot.setText("");
    tubyr.setText("");
    tukemb.setText("");
    cbkode.setSelectedItem("PILIH");
}

private void cbkodeActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    brg.setKd(cbkode.getSelectedItem().toString());
    brg.setSeleksiKd();
    nb.setText(brg.getNm());
    thrG.setText(Double.toString(brg.getHrg()));
    tjml.requestFocus();
}

```

```

public void setNm(String val){
    nm=val;
}
public String getNm(){
    return nm;
}
public void setHrg(Double val){
    hrg=val;
}
public double getHrg(){
    return hrg;
}

public void setJml(int val){
    jml=val;
}
public int getJml(){
    return jml;
}
public double getTot(){
    ttl=hrG*jml;
    return (ttl);
}
public void setUb(double val){
    ub=val;
}
public double getUb(){
    return ub;
}
public double getUk(){
    return (ub-ttl);
}
}

```

Kode Program Form

```
import java.awt.event.KeyEvent;
import javax.swing.JOptionPane;

public final class FormBarang extends javax.swing.JFrame {
    Barang brg=new Barang();
    /** Creates new form FormBarang */
    public FormBarang() {
        initComponents();

        cbkode.addItem("PILIH");
        cbkode.addItem("B001");
        cbkode.addItem("B002");
        cbkode.addItem("B003");

        nonaktif();
        bersih();
    }

    private void tjmlKeyPressed(java.awt.event.KeyEvent evt) {
        // TODO add your handling code here:
        if(evt.getKeyCode()==KeyEvent.VK_ENTER)
        {
            brg.setJml(Integer.parseInt(tjml.getText()));
            ttot.setText(Double.toString(brg.getTot()));
            tubyr.grabFocus();
        }
    }

    private void tubyrKeyPressed(java.awt.event.KeyEvent evt) {
        // TODO add your handling code here:
        if(evt.getKeyCode()==KeyEvent.VK_ENTER)
        {
            brg.setUb(Double.parseDouble(tubyr.getText()));
            tukemb.setText(Double.toString(brg.getUk()));
            if(brg.getUb()<brg.getTot()){
                JOptionPane.showMessageDialog(null, "Pembayaran kurang!!",
                    "Warning",JOptionPane.OK_OPTION);
                tukemb.setText("");
                tubyr.setText("");}
            else{
                return;
            }
        }
    }

    private void bcloseActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        int x;
        x=JOptionPane.showConfirmDialog(null, "Yakin mau keluar??","Konfirm",
            JOptionPane.YES_NO_OPTION);
        if (x==JOptionPane.YES_OPTION)
            dispose();
        else
            return;
    }

    void aktif(){
        cbkode.setEnabled(true);
        tjml.setEnabled(true);
        tubyr.setEnabled(true);
    }

    void nonaktif(){
        cbkode.setEnabled(false);
        nb.setEnabled(false);
        thrgr.setEnabled(false);
        tjml.setEnabled(false);
        ttot.setEnabled(false);
        tubyr.setEnabled(false);
        tukemb.setEnabled(false);
    }
}
```

Pada class Barang.java terdapat tambahan method yaitu setSeleksiKd() yang berfungsi untuk menyeleksi kode barang mana yang kita pilih pada saat program di run. Method penyeleksian harus ditambahkan jika sesuai dengan keperluan program sehingga menghasilkan keputusan dan statement.

Didalam method FormBarang() terdapat method initComponents(). Penambahan item dapat kita letakkan setelah method initComponents(). Method initComponents() berfungsi untuk menampilkan object-object yang ada di form dan segala atributnya. Oleh karena itu, source code yang ingin programmer jalankan setelah form tampil, harus diletakkan di dalam method ini. Dapat dilihat dari contoh source code disamping. Dengan method addItem() maka JComboBox cbKode akan ditambahkan isian item sesuai dengan jumlah yang diinginkan oleh programmer. Disitu terdapat juga method nonaktif() dan bersih(), dimana pada saat form dijalankan, beberapa komponen form tidak aktif dan isianya akan dihilangkan.

```
public FormBarang() {  
    initComponents();  
  
    cbkode.addItem("PILIH");  
    cbkode.addItem("B001");  
    cbkode.addItem("B002");  
    cbkode.addItem("B003");  
  
    nonaktif();  
    bersih();  
}
```


BAB XI

PENYELESAIAN CONTOH PROJECT

11.1. Kopetensi Dasar

Pada pembahasan Bab 11 ini Membuat form dan implementasi logika dengan program form dan program class pada bahasa Pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa dapat:

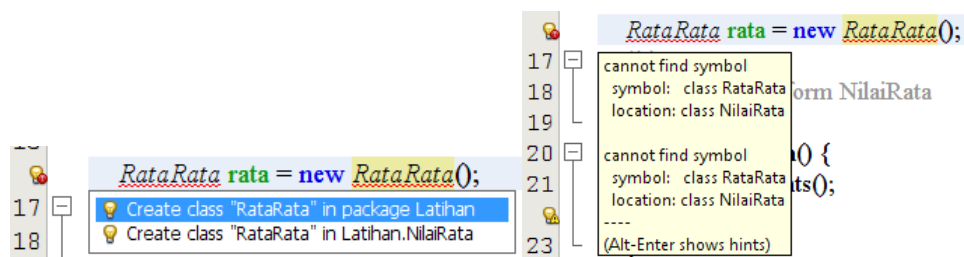
1. Review Praktek OOP dalam Program Netbeans dan Presentasi Studi Kasus
2. Review Praktek OOP dalam Program Java dan Presentasi Studi Kasus program

11.2. Pembuatan Form Dan Debugging Error

Debugging error sangat penting dikuasai oleh seorang programmer dengan jenis bahasa pemrograman apapun. Begitu pula dengan bahasa pemrograman berbasis objek Java. Jika kita menggunakan aplikasi Netbeans dalam pembuatan programnya, Netbeans biasanya memberikan tanda error disebelah kiri area pengetikkan source code. Tanda error tersebut dapat berupa ,



biasanya jika kita sorot icon error tersebut maka akan muncul tooltips keterangan kesalahannya. Dan jika kita klik icon error tersebut maka akan muncul solve option untuk mengatasi kesalahan dari satu baris source code.



Gambar solve option dan tooltips error

Dari gambar di atas, terlihat ada kesalahan pada satu baris source code. Tooltips error pada gambar diatas memberi tahu kesalahannya adalah “cannot find symbol”. Kesalahan ini biasa terjadi pada saat programmer salah dalam mengetikkan variable, nama class, nama method, dll. Pada kasus diatas, kesalahannya adalah java tidak dapat menemukan symbol (bisa jadi sebuah nama atau semacamnya) class RataRata.

Satu baris source code diatas dimaksudkan untuk membuat sebuah instansiasi objek dari class Ratarata.java, dimana formatnya adalah “namaClass objek baru=new namaClass();”. Akan tetapi pengetikkan class Ratarata diatas salah, karena tidak sesuai dengan nama class yang dimaksud. Pada saat anda membuat sebuah instansiasi objek dari sebuah class, makanama class harus diketikkan sama (case sensistive). Oleh karena itu pengetikkan yang benar adalah:

Seorang programmer harus dapat menganalisa kesalahan-kesalahan yang terjadi pada saat membuat program. Tetntu saja harus dapat mengatasinya juga. Dengan begitu akan mengurangi kesalahan-kesalahan yang akan terjadi di pembuatan program berikutnya.

Berikut adalah contoh kasus dimana terdapat kesalahan pada source code-nya dan tugas anda adalah menganalisa dan memperbaiki kesalahannya.

Buat Program dengan Layout sbb :

PROGRAM PERUMAHAN

DEVELOPER PERUMAHAN
PT. RUMAH MURAH MERIAH

Data Rumah

Lokasi

Harga Dasar

Fasilitas Rumah

☐ Jogging Track Rp. 100.000,00

☐ Swimming Pool Rp. 200.000,00

☐ Gymnasium Rp. 300.000,00

Cara Pembayaran

☐ BTN

☐ Other Bank

Total

HITUNG BERSIH KELUAR

Properties :

ComboBox → name : cmblok

textField2 → name : tbtn

textField1 → name : thr

textField3 → name : tother

CheckBox1 → name : cbjog

CheckBox2 → name : cbsw

CheckBox3 → name : cbgy

RadioButton1 → name : rbtn

RadioButton2 → name : rbob

Catatan : jika membuat radiobutton harus menambahkan objek ButtonGroup → beri nama : bg, Kemudian ubah masing2 properties di rbbtn → buttongroup → bg, rbbob → buttongroup → bg

Ketentuan :

1. Lokasi dan Harga

<u>Blok</u>	<u>Harga</u>
Blok A	Rp.2,000,000
Blok B	Rp.3,000,000
Blok C	Rp.4,000,000

2. Fasilitas dan Biaya

Jogging Track	= Rp. 100,000
Swimming Pool	= Rp. 200,000
Gymnasium	= Rp. 300,000

3. Cara Pembayaran

BTN	= Rp. 10,000
Other Bank	= Rp. 50,000

4. Buat metod aktif, nonaktif dan bersih

5. Buat class dengan nama **Rumah.java**

6. Buat Form dengan nama **FormRumah.java**

7. Total = Harga dasar + Fasilitas + cara bayar

Source code class Rumah.java:

```
public class Rumah {  
    public String lokasi,jo,swim,gym,cara;  
    public double hrg=0,hrg_j=0,hrg_s=0,hrg_g=0,biaya=0,total=0;  
  
    public void setLokasi(String val){  
        lokasi=val;  
    }  
  
    public String getLokasi(){  
        return lokasi;  
    }  
  
    public double getHrg(){  
        return hrg;  
    }  
}
```

```

}
public void setSeleksiLok(){
if(lokasi.equalsIgnoreCase("BLOK A")){
hrg="20000000";
} else if(lokasi.equalsIgnoreCase("BLOK B")){
hrg="30000000";
} else if(lokasi.equalsIgnoreCase("BLOK C")){
hrg="40000000";
} else {
hrg="0";
}
}
}

```

```

Public void setJog(String val){
Jog=val;
}

```

```

Public double getJog(){
If(jog.equalsIgnoreCase("Jogging track")){
Hrg_j=100000;
{else{
Hrg_j=0;
}
Return hrg_j;
}
}

```

```

Public void setSwim(String val){
Swim=val;
}

```

```

Public double getSwim(){
If(swim.equalsIgnoreCase("Swimming pool")){
Hrg_s=200000;
{else{
Hrg_s=0;
}
Return hrg_s;
}
}

```

```

Public void setGym(string val){
Gym=val;
}

```

```

Public double getGym(){
If(gym.equalsIgnoreCase("Gymnasium")){
Hrg_g=300000;
{else{
Hrg_g=0;
}
Return hrg_g;
}
}

```

```
}
```

```
Public void setCara(String val){  
Cara=val;  
}
```

```
Public String getCara(){  
Return cara;  
}
```

```
Public double getBiaya(){  
Return biaya;  
}
```

```
Public void setSeleksiCara(){  
If(cara.equalsIgnoreCase("BTN")){  
Biaya=10000;  
}else if(cara.equalsIgnoreCase("Other Bank")){  
Biaya=5000;  
}else{  
Biaya=0;  
}  
}
```

```
Public double getTotal(){  
Return(hrg+hrg_j+hrg_s+hrg_g+biaya);  
}
```

Source code class FormRumah.java:

```
Public class FormRumah extends javax.swing.JFrame{  
Rumah rmh=new Rumah();
```

```
Public FormRumah(){  
initComponents();  
nonaktif();  
bhitung.setEnabled(false);  
}
```

```
Void aktif(){  
Cmblok.setEnabled(true);  
Cbjog.setEnabled(true);  
Cbggy.setEnabled(true);  
Cbsw.setEnabled(true);  
rbbtn.setEnabled(true);  
rbob.setEnabled(true);  
}
```

```
Void bersih(){  
Cmblok.setSelectedItem("PILIH");  
Cbjog.setSelected(false);
```

```

Cbsw.setSelected(false);
Cbgj.setSelected(false);
Bg.clearSelection();
Thrg.setText("");
Tbtn.setText("");
Tother.setText("");
Ttot.setText("");
}

```

```

Void nonaktif(){
Cmblok.setEnabled(false);
Cbjog.setEnabled(false);
Cbgj.setEnabled(false);
Cbsw.setEnabled(false);
rbbtn.setEnabled(false);
rbob.setEnabled(false);
thrg.setEnabled(false);
tbtn.setEnabled(false);
ttot.setEnabled(false);
}

```

```

Private void cmblokActionPerformed(java.awt.event.ActionEvent evt){
Rmh.setLokasi(cmblok.getSelectedItem().toString());
Rmh.setSeleksiLok();
Thrg.setText(Double.toString(rmh.getHrg()));
}

```

```

Private void cbjogActionPerformed(java.awt.event.ActionEvent evt){
String j;
If(cbjog.isSelected()){
J=cbjog.getText();
Rmh.setJog(j);
Rmh.getJog();
}else{
J=" ";
Rmh.setJog(j);
Rmh.getJog();
}
}

```

```

Private void cbswActionPerformed(java.awt.event.ActionEvent evt){
String s;
If(cbsw.isSelected()){
s=cbsw.getText();
Rmh.setSwim(s);
Rmh.getSwim();
}else{
s=" ";
Rmh.setSwim(s);
Rmh.getSwim();
}
}

```

```
}  
}
```

```
Private void cbgyActionPerformed(java.awt.event.ActionEvent evt){  
String g;  
If(cbgy.isSelected()){  
g=cbgy.getText();  
Rmh.setGym(g);  
Rmh.getGym();  
}else{  
g=” “;  
Rmh.setGym(g);  
Rmh.getGym();  
}  
}
```

```
Private void rbbtnActionPerformed(java.awt.event.ActionEvent evt){  
Rmh.setCara(rbbtn.getText());  
Rmh.setSeleksiCara();  
Tbtn.setText(Double.toString(rmh.getBiaya()));  
Tother.setText(“ “);  
}
```

```
Private void rbobActionPerformed(java.awt.event.ActionEvent evt){  
Rmh.setCara(rbob.getText());  
Rmh.setSeleksiCara();  
Tother.setText(Double.toString(rmh.getBiaya()));  
Tbtn.setText(“ “);  
}
```

```
Private void bhitungActionPerformed(java.awt.event.ActionEvent evt){  
Thrg.setText(Double.toString(rmh.getHrg()));  
Ttot.setText(Double.toString(rmh.getTotal()));  
}
```

```
Private void bbersihActionPerformed(java.awt.event.ActionEvent evt){  
Bersih();  
Aktif();  
Bhitung.setEnabled(true);  
}
```

Buatlah source code untuk button keluar....!!!
Contoh kasus Form Penjualan Motor

Buatlah sebuah tampilan form program penjualan sepeda motor seperti di bawah ini :

APLIKASI PENJUALAN MIOTOR

FORM PENJUALAN MOTOR
DEALER MAJU MOTOR

NAMA PEMBELI

DATA MOTOR

MERK MOTOR

JENIS MOTOR ☐ BEBEK ☐ MATIC ☐ SPORT

HARGA MOTOR

JENIS BAYAR

DATA HARGA MOTOR

DISKON

TOTAL BAYAR

Properties :

JtextField1 → name : ttmpembeli

JtextField2 → name : TDISKON

JtextField2 → name : THARGA

JtextField3 → name : tother

JComboBox1 → name : CBMERK

JButton1 → name : BDISKON

JComboBox2 → name : CBJENIS

JButton2 → name : BBARU

JRadioButton1 → name : R1

JButton1 → name : BDISKON

JRadioButton2 → name : R2

JButton1 → name : BSELESAI

JRadioButton3 → name : R3

JButtonGroup1 → name : bg

Ketentuan :

1. Harga Motor

Jika merk motor Honda dan Jenis motor Bebek maka harga 10000000

Jika merk motor Honda dan Jenis motor Matic maka harga 13000000

Jika merk motor Honda dan Jenis motor Sport maka harga 16000000

Jika merk motor Yamaha dan Jenis motor Bebek maka harga 12000000

Jika merk motor Honda dan Jenis motor Bebek maka harga 10000000

Jika merk motor Honda dan Jenis motor Bebek maka harga 10000000

Jika merk motor Honda dan Jenis motor Bebek maka harga 10000000

Jika merk motor Honda dan Jenis motor Bebek maka harga 10000000

Jika merk motor Honda dan Jenis motor Bebek maka harga 10000000

Jika merk motor Honda dan Jenis motor Bebek maka harga 10000000

2. Jenis Bayar : Cash atau Credit

3. Diskon

Jika Jenis motor Bebek = 5% dari Harga

Jika Jenis motor Matic = 3% dari Harga

Jika Jenis motor Sport = 1% dari Harga

4. Buat metod aktif, nonaktif dan bersih

5. Buat class dengan nama **Motor.java**

6. Buat Form dengan nama **FormMotor.java**

7. Total = Harga - Diskon

BAB XII

CONTOH KASUS

12.1. Kopetensi Dasar

Pada pembahasan Bab 12 ini penulis mengajak mendiskusikan mengenai studi kasus yang yang harus diselesaikan dengan Bahasa Pemrograman Java. Kopetensi dasar secara umum, agar mahasiswa atau pembaca bisa menganalisa dan menyelesaikan dengan bahasa pemrograman Java. Mahasiswa diharapkan dapat :

1. Penerapan Program Java dalam Studi Kasus
2. Mahasiwa dapat mempraktekkan Program sesuai permasalahan pada contoh kasus

12.2. Menganalisa contoh kasus I

Food court “Oishi ne” merupakan rumah makan yang menyediakan menu makanan Jepang, dimana masing-masing menu makanan disediakan dalam bentuk paket yang memiliki nama menu paket dan harga.

Untuk menu minuman, food court “Oishi ne” menyediakan air mineral, berbagai macam juice dengan harga yang sama, dan es teh manis.

Agar lebih dekat dengan pelanggan dan meningkatkan pelayanan, food court “Oishi ne” memberikan pelayanan tambahan, yaitu delivery dan take away.

Proses program:

1. Untuk memulai program, kasir harus menekan tombol bersih agar inputan transaksi sebelumnya hilang dan semua objek dalam program siap digunakan.
2. Pelanggan memilih paket makanan yang akan dibeli. Dengan memilih salah satu paket maka nama menu dan harga paket akan tampil di masing-masing textfield.
3. Pelanggan menentukan jumlah menu yang dibeli, dan tugas kasir adalah menginput jumlah beli sesuai keinginan pelanggan. Dengan menekan tombol enter maka total harga yang harus dibayar oleh pelanggan tampil sesuai dengan kalkulasi harga paket dan jumlah yang dibeli.
4. Sebelum transaksi selesai, pelanggan diperbolehkan untuk memilih minuman. Minuman dapat dipilih lebih dari satu, misal pelanggan memilih minuman air mineral gelas dan juice 1 gelas.
5. Ketika diinput jumlah minumannya, maka total bayar akan bertambah sesuai dengan hasil kalkulasi pembelian minuman. Contoh: jika total bayar 1000, dan pelanggan membeli air mineral 2 gelas dengan total harga 500, maka total bayar menjadi 1500.

6. Begitu juga dengan jenis pelayanannya. Untuk pelayanan delivery, food court “Oishi ne” memberikan tarif 10% dari total bayar. Sedangkan untuk take away tidak dikenakan tarif. Tariff pelayanan akan muncul otomatis di sebuah textfield.
7. Selanjutnya kasir akan menekan tombol proses, sehingga total bayar akan dijumlahkan dengan tarif pelayanannya.
8. Kasir menginput uang bayar yang diberikan pelanggan, jika uangnya melebihi dari total yang harus dibayarkan, maka akan muncul otomatis uang kembaliannya.
9. Jika transaksi selesai maka kasir dapat menekan tombol keluar, dimana pada saat menekan tombol tersebut, program mengkonfirmasi kasir dengan memberikan pertanyaan “Apakah anda yakin ingin menutup aplikasi?”. Jika kasir memilih Yes maka program ditutup, dan sebaliknya.

1. Pembuatan program

Buatlah program sesuai dengan analisa dan proses program yang sudah dijelaskan diatas. Dengan penjelasan tersebut anda dapat menentukan obyek swing apa saja yang akan digunakan dalam pembuatan form. Begitu juga dengan subclass yang akan dibuat, anda dapat menentukan variable apa saja yang harus dimasukkan ke dalam subclass tersebut termasuk method-methodnya.

Anda dapat menentukan sendiri harga-harga dari menu minuman dan makan serta tarif pelayanannya.

12.3. Menganalisa contoh kasus II

Contoh kasus ini tentang Klinik “Sehat Selalu”. Klinik “Sehat selalu” memiliki beberapa dokter spesialis. Jam prakteknya mulai dari jam 08.00-15.00 dan 16.00-23.00. Klinik ini memiliki tiga spesialis dokter yaitu umum, gigi, dan saraf. Masing-masing dokter mempunyai jam praktek yang berbeda-beda.

Nomor pasien akan dikenali jika mereka sudah terdaftar. Dengan kata lain, program ini hanya dapat melakukan transaksi dengan pasien yang sudah terdaftar dan memiliki nomor pasien.

Pasien dapat saja melakukan rawat inap jika diharuskan seperti itu. Pasien rawat inap disediakan ruang/kamar inap untuk beberapa hari saja. Kamar inap terdiri dari 4 kamar dengan harga inap yang berbeda-beda.

Pasien dapat melakukan konsultasi ke dokter dan berobat (obat), dimana tariff layanan tersebut berbeda. Layanan tersebut juga dapat dilakukan oleh pasien secara bersamaan. Setiap pasien yang datang ke klinik tersebut dan melakukan transaksi akan dikenakan biaya administrasi.

Proses program:

1. Pasien datang ke bagian administrasi untuk melakukan transaksi. Selanjutnya bagian administrasi akan menginput nomor pasien. Dengan menekan tombol enter maka nama pasien pun muncul setelah nomor pasien diinput oleh bagian administrasi.
2. Kasir akan memilih jenis spesialis terlebih dahulu sebelum memilih nama dokter. Nama-nama dokter yang akan muncul didaftar nama dokter akan tampil setelah jenis spesialis dipilih. Misalnya bagian administrasi memilih jenis spesialis syaraf, maka akan muncul beberapa nama dokter yang memiliki jenis spesialis tersebut.
3. Bagian administrasi memilih dokter sesuai dengan surat rujukan dari pasien lalu dengan otomatis akan muncul waktu/jam praktek dari dokter yang dipilih tadi.
4. Bagian administrasi akan menginput lama inap yang diikuti dengan pemilihan ruang/kamar inap. Dengan memilihnya ruang/kamar inap, maka akan muncul harga inap per harinya dan diskon yang diberikan untuk pasien. Diskonnya tergantung dari jenis kamar yang dipilih.
5. Bagian administrasi wajib memilih pembayaran yang harus dilakukan oleh pasien, entah itu biaya administrasi, konsultasi dokter dan obat.
6. Semua harga yang tampil baik itu kamar inap dan pembayaran akan tampil di textfield total bayar dengan cara menekan tombol hitung pada form. Jika transaksi sudah selesai dilakukan, kasir dapat menekan tombol tutup untuk menutup aplikasi.

1. Pembuatan program

Buatlah program sesuai dengan analisa dan proses program yang sudah dijelaskan di atas. Dengan penjelasan tersebut anda dapat menentukan obyek swing apa saja yang akan digunakan dalam pembuatan form. Begitu juga dengan subclass yang akan dibuat, anda dapat menentukan variable apa saja yang harus dimasukkan ke dalam subclass tersebut termasuk method-methodnya. Subclass untuk contoh kasus ini baiknya dibedakan menjadi 3 subclass, misal subclass klinik, dokter, dan pasien. Semua subclass akan diproses di main class yaitu di FormTransaksi Klinik “Sehat Selalu”.

Anda dapat menentukan sendiri harga-harga dari biaya administrasi, pembayaran, dan ruang/kamar inap.